REPORT DOCUMENTATION PAGE	*Form Approved OMB No. 0704-0188*

Public reporting burden for this collection of information is estimated to average 1 hour per response, including the time for reviewing instructions, searching existing data sources, gathering and maintaining the data needed, and completing and reviewing this collection of information. Send comments regarding this burden estimate or any other aspect of this collection of information, including suggestions for reducing this burden to Department of Defense, Washington Headquarters Services, Directorate for Information Operations and Reports (0704-0188), 1215 Jefferson Davis Highway, Suite 1204, Arlington, VA 222024302. Respondents should be aware that notwithstanding any other provision of law, no person shall be subject to any penalty for failing to comply with a collection of information if it does not display a currently valid OMB control number. PLEASE DO NOT RETURN YOUR FORM TO THE ABOVE ADDRESS.

1. REPORT DATE *(DD-MM-YYYY)*	2. REPORT TYPE	3. DATES COVERED *(From - To)*
03-11-2025	Technical Report – Monthly Status Report	3 October 2025 – 31 October 2025

4. TITLE AND SUBTITLE	5a. CONTRACT NUMBER
Phase I Final Report Contract Line Item: 0003: Technical Report SF24C-T003: Strengthening Space: Uniting Research and Innovation for PWSA	FA254125PB053
	5b. GRANT NUMBER
	5c. PROGRAM ELEMENT NUMBER

6. AUTHOR(S)	5d. PROJECT NUMBER
Bernard P. Zeigler, PhD (PI) zeigler@rtsync.com, Christian Koertje christian.koertje@rtsync.com, Paul Wach, PhD (Co-PI) paulw86@vt.edu	5e. TASK NUMBER
	5f. WORK UNIT NUMBER

7. PERFORMING ORGANIZATION NAME(S) AND ADDRESS(ES)	8. PERFORMING ORGANIZATION REPORT NUMBER
RTSync Corp. 6909 W Ray Rd STE 15-107 Chandler, AZ 85226	2025_SF-STTR24C-T003-PH1-03

9. SPONSORING / MONITORING AGENCY NAME(S) AND ADDRESS(ES)	10. SPONSOR/MONITOR'S ACRONYM(S)
Mr. Richard Hencke richard.b.hencke.civ@mail.mil Dr. Russell Teehan russell.f.teehan.civ@mail.mil Space Systems Command FA2541 SSC COMRCL SVS OFC SSC CMK CP 310 744 5804, 14850 CONFERENCE CENTER DR CHANTILLY, VA 20151	SSC
	11. SPONSOR/MONITOR'S REPORT NUMBER(S)

12. DISTRIBUTION / AVAILABILITY STATEMENT
Distribution authorized to U.S. Government Agencies only

13. SUPPLEMENTARY NOTES

14. ABSTRACT

This report for Phase I project "SF24C-T003: Strengthening Space: Uniting Research and Innovation for PWSA", describes progress made on the proof-of-concept system design, use case identification, and feasibility demonstration researched and developed during Phase I. We propose Discrete Event System Specification (DEVS) System theory-based framework for mission engineering and analysis to support Proliferated Warfighter Space Architecture (PWSA) performance modeling. This framework is aimed at highlighting the intricate dynamics between space systems and their terrestrial counterparts across air, land, and sea domains with a specific focus on tranche-based mission engineering and analysis. Our objective centers on developing a prototype that accounts for the dynamic intricacies of space-enabled systems and the need to effectively and efficiently transition between planned PWSA development tranches.

15. SUBJECT TERMS Proliferated Warfighter Space Architecture, performance modeling, resilience, digital transformation, model-based systems engineering, modeling & simulation, tranche-based, applied systems engineering theory

16. SECURITY CLASSIFICATION OF:			17. LIMITATION OF ABSTRACT	18. NUMBER OF PAGES	19a. NAME OF RESPONSIBLE PERSON
a. REPORT	b. ABSTRACT	c. THIS PAGE	UU	63 including cover	SSC
U					19b. TELEPHONE NUMBER *(include area code)*

Phase I Status Report

Contract Number: FA254125PB053

Line Item: 0003

Reporting Period: October 3, 2025 – October 31, 2025

SF24C-T003: Strengthening Space: Uniting Research and Innovation for PWSA (Proliferated Warfighter Space Architecture).

Prepared for:

Space Systems Command

FA2541 SSC COMRCL SVS OFC SSC CMK

CP 310 744 5804, 14850 CONFERENCE CENTER DR

CHANTILLY, VA 20151

TPOC:

Mr. Richard Hencke	richard.b.hencke.civ@mail.mil
Dr. Russell Teehan	russell.f.teehan.civ@mail.mil

SBIR/STTR DATA RIGHTS

Topic Number: SF24C-T003

Contract No.: FA254125PB053

Contractor Name: RTSync Corp.

Contractor Address: 6909 W Ray Rd Suite 15-107 Chandler, AZ 85226

Contractor Emails: zeigler@rtsync.com, dhkim@rtsync.com, paulw86@vt.edu

Prepared By RTSync Corp.

11/3/2025

Executive summary

This report for STTR Phase I project "SF24C-T003: Strengthening Space: Uniting Research and Innovation for PWSA (Proliferated Warfighter Space Architecture)" summarizes progress in developing a proof-of-concept framework for performance modeling, mission engineering, and tranche-based analysis of space systems supporting the PWSA vision.

Staying true to the SDA emphasis on speed, delivery, and agility, our work aims to deliver Discrete Event System Specification (DEVS) System theory-based framework for mission engineering and analysis to support PWSA performance modeling. This framework is aimed at highlighting the intricate dynamics between space systems and their terrestrial counterparts across air, land, and sea domains with a specific focus on tranche-based mission engineering and analysis. Our objective centers on developing a prototype that accounts for the dynamic intricacies of space-enabled systems and the need to effectively and efficiently transition between planned PWSA development tranches. The prototype will harness the synergy of RTSync's collaborative modeling and simulation (M&S) environment, with the high-fidelity M&S capabilities.

Phase I focused on four core technical objectives:

1. **DEVS-PWSA Framework Formulation –** Defined a mathematically underpinned modeling structure for PWSA performance assessment, emphasizing reusability, scalability, and cross-domain applicability.

2. **Algorithm Development –** Designed and implemented algorithms for math-based mapping (i.e., homomorphism) between physics models and probabilistic Markov representations, supported by ontology-driven graph databases (Neo4j and ArcadeDB).

3. **Test Environment Development –** Constructed a prototype environment combining DEVS models, graph database infrastructures, and the Protégé ontology editor to support semantic consistency and automated data exchange across model layers.

4. **Feasibility Demonstration** – Executing a solar-flare disruption use case to validate the framework's ability to quantify dynamic performance degradation and demonstrate equivalence mapping (i.e., homomorphism) between physics-based models and key performance indicators such as probability of connection.

The Phase I prototype successfully demonstrated automated equivalence mapping (i.e., homomorphism), ontology-linked model representation, and interoperability between simulation environments and knowledge graphs. These capabilities lay the groundwork for future integration with agentic AI, enabling automated reasoning, model discovery, and digital test and evaluation (dT&E) for mission resilience under dynamic space conditions.

Next Steps

Phase II will extend the framework to incorporate additional PWSA layers (e.g., tracking), expand software interoperability (e.g., Cameo, MATLAB, Ansys STK, Rosetta), and introduce agentic AI orchestration for adaptive model synthesis and mission-level decision support. The enhanced architecture will promote a unified digital engineering ecosystem supporting SDA's evolving tranche-based development strategy.

Table of Contents

Executive summary 2
Table of Contents 3
Acronyms 4
1 Introduction 5
2 Work planned and accomplished 6
2.1 Overview of work planned for Phase I 6
2.2 Overview of work accomplished 6
3 Task 1: DEVS Modeling Framework for PWSA Performance Modeling Formulation 7
3.1 Developing Tools for Automating Morphism Mappings Between DEVS Models 7
3.2 Markov Models for Establishing Effects of Solar Flare Disruption 10
4 Task 2: Algorithm Development 11
5 Task 3: Test Environment Development 13
6 Task 4: Feasibility Demonstration 14
6.1 Morphism Modeling Application 17
7 Future Work Planned 18
References 21
Appendix A: Background on MBSE and M&S 24
DEVS background 24
MBSE and Science-Based Based Verification, Validation, And Accreditation (VV&A) Process 25
Appendix B: Background on Homomorphisms and Parameter Morphisms 26
Appendix C: Ontology and Application to PWSA 28
Appendix D: Graph Databases and Algorithms 32
Appendix E: Agentic AI Exploration and Considerations for Future Phases 36
Appendix F: Presentation Slides of the Monthly Status Meeting 39

Acronyms

DB	Database
DE	Digital Engineering
DEVS	Discrete Event Systems Specification
DoD	Department of Defense
dT&E	Digital Test & Evaluation
EF	Experimental Frame
GAO	Government Accountability Office
KPI	Key Performance Indicator
LEO	Low-Earth Orbit
LLM	Large Language Model
MBSE	Model-Based Systems Engineering
ME	Mission Engineering
MSA	Modeling, Simulation, and Analysis
MoP	Measures of Performance
M&S	Modeling and Simulation
NTN	Non-Terrestrial Network
OCT	Optical Communication Terminals
PT&E	Phased Test and Evaluation
PWSA	Proliferated Warfighter Space Architecture
RF	Radio Frequency
SDA	Space Development Agency
SE	System Engineering
SES	System Entity Structure
SoS	System of Systems
T&E	Test & Evaluation
V&V	Verification and Validation
VV&A	Verification, Validation, and Accreditation

1 Introduction

The Space Development Agency (SDA) is founded on principles of speed, delivery, and agility. Current approaches reduce the rapid turnaround necessary to achieve these foundational principles. For example, it is challenging to leverage knowledge from past tranches for tranches in development or expected in the future due to models being constructed that lack reusability and interoperability. A wide range of simulation models have been developed for PWSA performance modeling in relation to test and evaluation (T&E) among other contexts. Currently however, a framework is lacking by which existing models can be rapidly integrated to address performance questions as they arise. Moreover, the current methodology does not have sufficient flexibility to accommodate new models that are expected to come online and older models that will be upgraded and replaced. Moreover, scalability is required to deal with questions that range from those focusing on individual payloads to those concerning full constellation performance in which payloads must be only one component in the aggregation. Finally, the existing wide range of fidelity levels and computational burdens requires justified means for tradeoff balances between accuracy and computation times for reasonable analysis products.

We proposed Discrete Event System Specification (DEVS) [1] System theory-based framework for mission engineering & analysis to support Proliferated Warfighter Space Architecture (PWSA) performance modeling. This framework is aimed at highlighting the intricate dynamics between space systems and their terrestrial counterparts across air, land, and sea domains with a specific focus on tranche-based mission engineering and analysis. Our objective centers on developing a prototype that accounts for the dynamic intricacies of space-enabled systems and the need to rapidly transition between tranches. Accordingly, we propose DEVS system theory based framework for PWSA performance modeling (DEVS-PWSA) to meet the requirements summarized below:

1. **Rapid Integration**: We can quickly incorporate existing tools into the DEVS-PWSA framework, supporting the proposed iterative development.

2. **Flexibility**: The framework easily accommodates new models and interactions as needs evolve, allowing for updates for new tranches/launches.

3. **Scalability**: From individual payloads to full constellation performance, this approach scales effectively, supporting both capability-specific and constellation-level analyses.

4. **Multi-fidelity Analysis**: Multiplicity of models at selectable scopes and resolution levels helps balance computational efficiency with accuracy requirements to meet diverse analyst demands.

The framework is proposed within the vision of the SDA's tranche-based constellation build-up. For example, one tranche may use optical communication, while a subsequent tranche may use RF communication (e.g., Link 16). How do we leverage the models and analysis conducted on tranche 1 to inform transition to tranche 2? We proposed to leverage RTSync Corp's advanced modeling and simulation (M&S) methodologies and Virginia Tech National Security Institute's (VT-NSI's) excellence in areas of test and evaluation, cyberphysical systems, and mission engineering to navigate the complexities of space-enabled missions. A principal challenge is deciphering the intricate multi-dimensional structures arising from interconnected agent collaborations. Our alliance comprises RTSync's M&S computational environment (MS4 Me [7,8]) and Collaborative Architecture Model-level Integration (CAMLI), sponsored by Missile Defense Agency's (MDA) SBIR programs. In collaboration, VT-NSI contributes its capabilities and expertise in aerospace and defense, mission engineering, including test and evaluation.

The importance of this endeavor is further underscored by the U.S. forces' reliance on the PWSA performance modeling, particularly as space-based technologies continue to evolve. The rapid advancements in these technologies necessitate agile and timely responses to threats. Our proposed synthesis and evaluation mechanisms are critical in clarifying requirements, directing design strategies,

facilitating cost-benefit analyses, and adapting operational tactics, especially in the context of time-sensitive challenges [2,3,4].

2 Work planned and accomplished

2.1 Overview of work planned for Phase I

The top-level tasks planned were:

(1) **DEVS Modeling Framework for PWSA Performance Modeling Formulation**

(2) **Algorithm Development**

(3) **Test Environment Development**

(4) **Feasibility Demonstration**

2.2 Overview of work accomplished

Staying true to the Space Development Agency emphasis on Space Technology, *our proposed work plan aims to develop sophisticated modeling, simulation, and analysis utilities*. These tools are envisioned to *elucidate the intricate dynamics* between space mechanisms and their terrestrial counterparts *for the evaluation of space-based mission engineering and analysis.*

Primary Objectives of PHASE I:

- Elaborate a concept for a prototype, emphasizing the application of modeling and simulation (M&S) to develop DEVS-PWSA to analyze and evaluate war-fighter relevant scenarios.
- Enhance and utilize the existing RTSync M&S infrastructure and software, leveraging its System of Systems (SoS) coupling capabilities
- Enhance and utilize the existing VT-NSI infrastructure in conjunction with RTSync infrastructure.
- Define/Concretize use case(s).
- Conduct analysis of a network node loss scenario.
- Demonstrate/Illustrate the technical feasibility to align with the specified objective for one use case.

Task Plan outline:

1. **DEVS Modeling Framework for PWSA Performance Modeling Formulation** (RTSync & VT-NSI Collaborative Task): Define the goals and criteria that the models need to satisfy
2. **Algorithms Development** (VT-NSI Responsibility): Create or prove use of existing algorithms between family of mission engineering and analysis models, which includes parameter mapping.
3. **Test Environment Development** (RTSync & VT-NSI Collaborative Task): Design and implement a conducive environment for rigorous model testing with DEVS-PWSA.
4. **Feasibility Demonstration** (RTSync & VT-NSI Collaborative Task): Validate the approach and demonstrate its applicability in real-world scenarios.

This work is performed within the context of Model Based System Engineering (MBSE) and its reliance on Modeling and Simulation (M&S), for which a brief background is given in Appendix A.

Model	Model Feature	Parameters and Relations
PWSA Operations Model	• Describes operation at the System of Systems level with component systems such as Satellite Operations, Mission Processing, Battle Management, Ground Support, and Communication Network. • Component systems provide specific services and engage in message exchanges such as mission directives and tasking, mission status updates and resource allocation.	
Base Model	Physics-based characterization of constellation structure and environment	Conceptual comprehensive system representation
Stochastic Connectivity	Predicts optimal message transmission paths under physical conditions mapped from Base Model with variance in design parameters	Random variables for connection parameters. E.g. PathDistance = N(mean, sigma)
Lumped Connectivity Model	Predicts expected message transmission paths under physical conditions mapped from Base Model	Utilizes lumped (average) values for connection variables E.g. PathDistance = <PathDistance>
Markov Model	• Stochastic model predicts SoS level statistics concerning message transmission quality (connectivity and latency)	Lumps connection variables into Probability of connection E.g. PathDistance not utilized

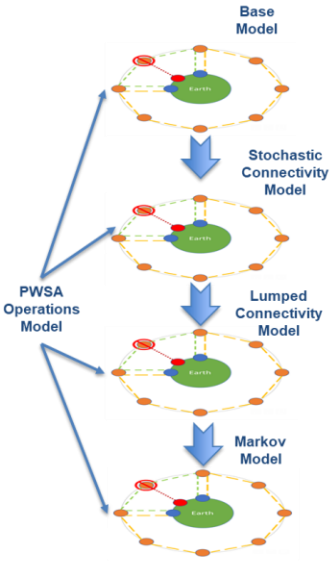


Figure 1: DEVS family of models spanning resolution for multi-resolution simulation framework.

3 Task 1: DEVS Modeling Framework for PWSA Performance Modeling Formulation

We previously introduced a methodology for developing a family of models across varying levels of resolution to support feasibility demonstrations (see Figure 1). The Base Model serves as a conceptual representation of the system, encompassing all pertinent behaviors as well as the necessary physical principles and parameters. At its lowest resolution, the DEVS Markov Model condenses complex physics and engineering processes related to satellite connectivity into simplified metrics, such as connection probabilities and mean latency values. Employing models with multiple resolutions offers distinct advantages for rapid decision-making: while high-resolution models provide greater detail and accuracy, they often require longer computation times; conversely, low-resolution models, when correct, can be executed more swiftly and used to inform immediate operational decisions as higher-resolution simulations are completed. In this context, we present an extension of the DEVS Markov model tailored to the solar flare experimental scenario.

3.1 Developing Tools for Automating Morphism Mappings Between DEVS Models

Having a family of models for V&V is a powerful part of our framework for MBSE, however it can be cumbersome to manage equivalent scenarios. To alleviate this, we are developing tools to ensure consistent parameter value mapping between versions of the Non-terrestrial Network (NTN) model. The premise of this framework is built on the homomorphism and parameter morphism concepts (see Appendix B for a brief background). The process starts with storing model information and morphism mappings in JSON file format. Generally, when mapping representations down from high resolution to low, the mapping will be many-to-one in which there is an expression that computes the target parameter value from the source parameter values. The expression that does the computation can be analytic, algorithmic, or derived from simulation and data. An example of this is a mapping from the Base Model to the DEVS Markov Model is sketched in Figure 2. This example demonstrates how the simplified model's probability of connection value might be influenced by the Base Model's physical parameter values. The scenario is constructed from the database.

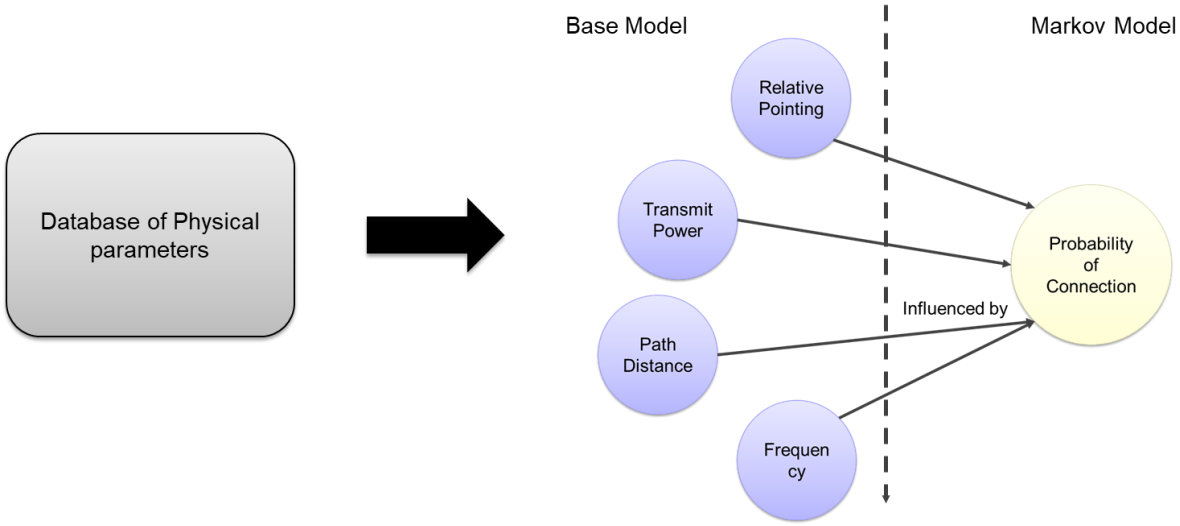


Figure 2: Mapping database parameter values from the base model scenario space to the lumped Markov model parameter values.

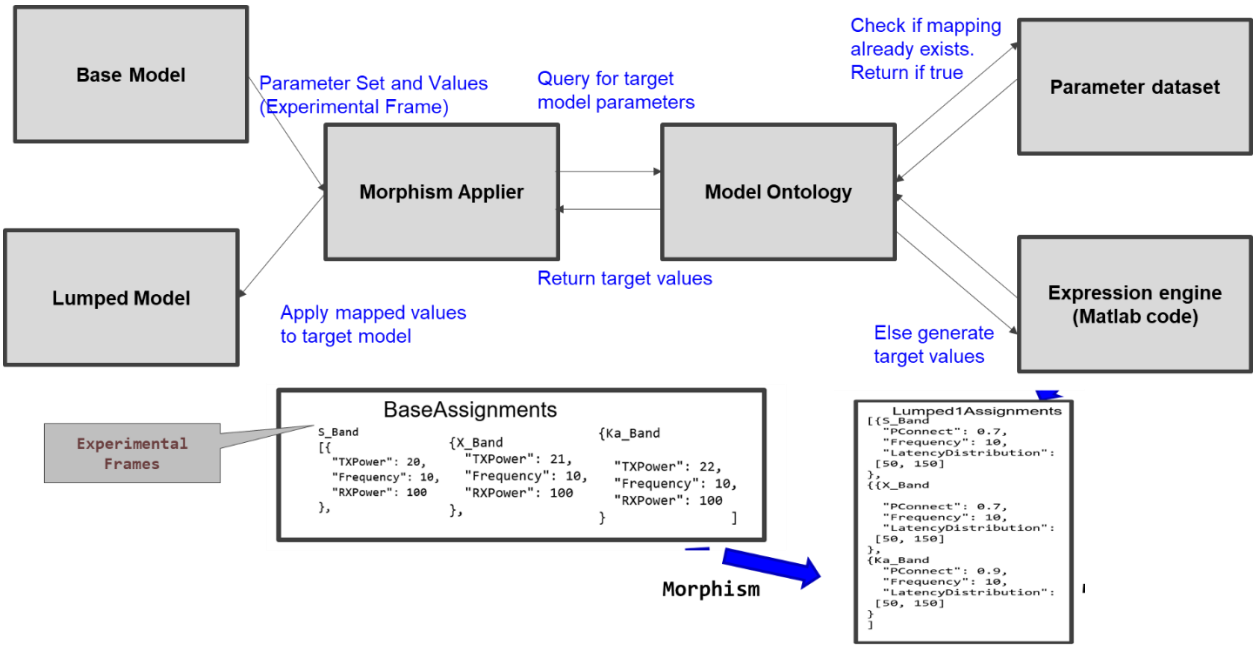


Figure 3: Data flow involved in taking Base model setting to a lumped model assignment.

The comprehensive workflow for generating a lumped model instance that is morphically equivalent to a base model scenario is illustrated in Figure 3. The process initiates with the definition of the experimental frame and the assignment of parameters for the base model. These inputs are then processed by the morphism applier within the simulation environment, which queries the model ontology to derive corresponding parameter values for the lumped model. The ontology retrieves these values either from an existing dataset or dynamically computes them using an expression engine. Once the mappings are established, the ontology returns the equivalent parameter configuration, which is applied to instantiate the lumped model. In summary, the system takes as input a base model, its parameter values, and a target

lumped model, and produces a morphically aligned parameter set for the lumped model. The resulting data structure is presented in JSON format in Figure 4.

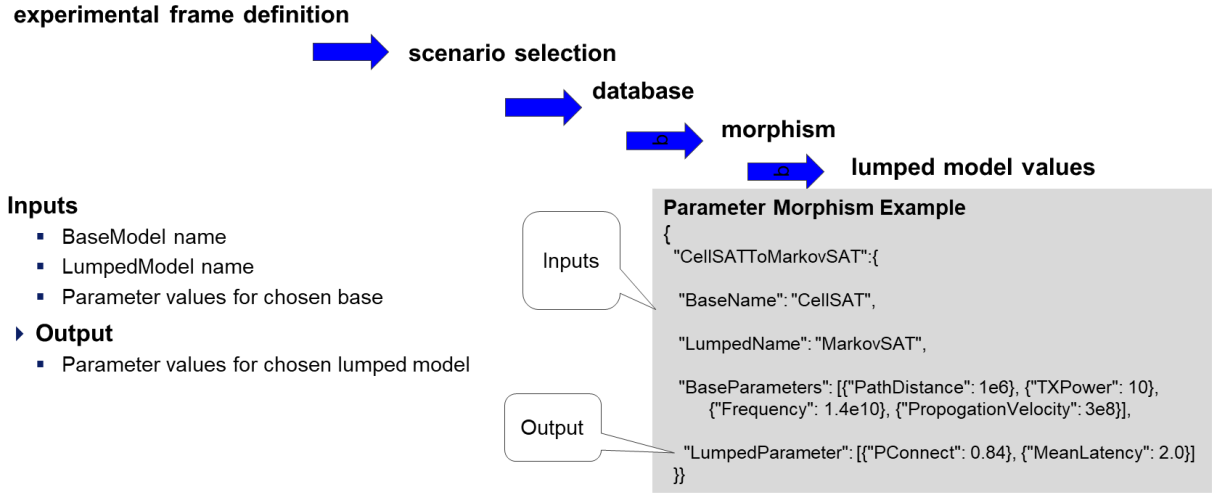


Figure 4: Details of the morphism and parameter values are stored in JSON format.

Within a family of models, it is possible to construct morphisms that establish formal relationships between varying levels of model resolution. For instance, a morphism can be defined from the base model to a lumped connectivity model, thereby linking detailed physical parameters to a simplified representation. As illustrated in Figure 5, this is referred to as the "BaseToConnectivity" morphism. The parameter set of the base model, which includes physics-based attributes, is mapped to a reduced set in the lumped model, retaining essential parameters while abstracting others. This lumped connectivity model can be further simplified into a DEVS Markov model, represented by the "ConnectivityToMarkov" morphism. These two mappings can be composed to form a higher-level morphism, "BaseToMarkov", which directly connects the base model to the Markov model. This compositional approach streamlines the transformation process and facilitates consistent parameter mapping across the entire model hierarchy.

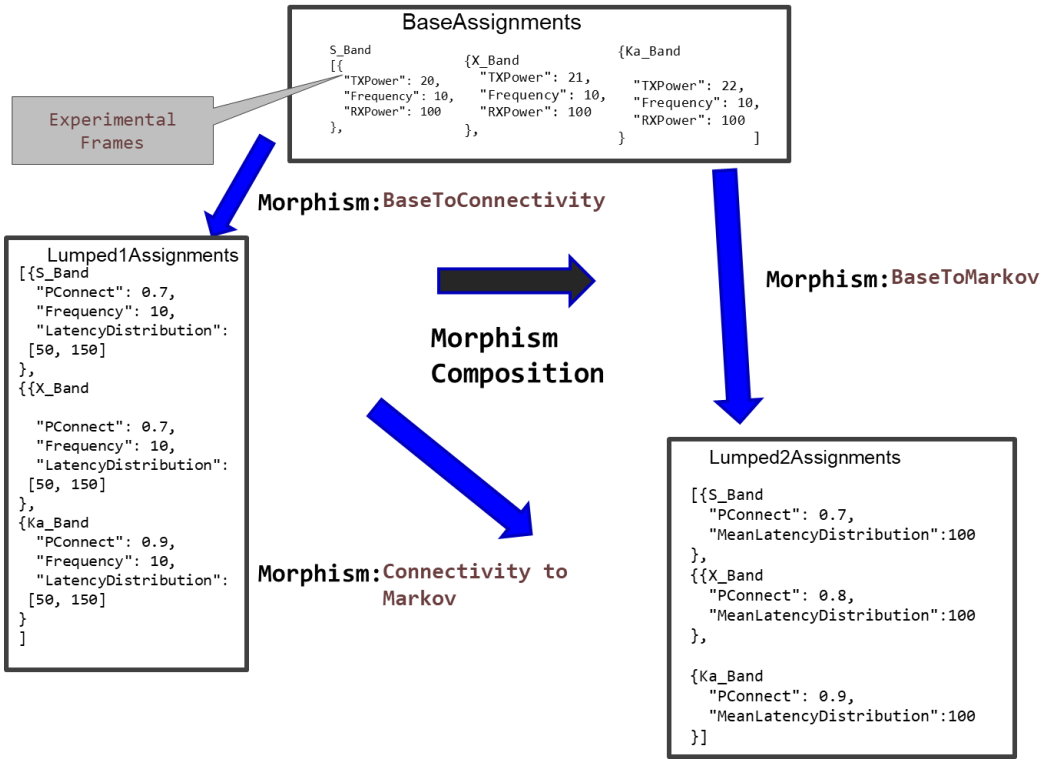


Figure 5: Construction of new morphisms via composition.

3.2 Markov Models for Establishing Effects of Solar Flare Disruption

To accommodate an incoming electronic disruption such as a solar flare, the DEVS satellite models need to update their internal state to reflect any disruptive effects such as damage to the electronic circuitry, interference with electromagnetic transmissions, among others. In a simplified model, latency degradation represents the satellite taking longer to transmit messages. The effect, represented in Figure 6, will be quantified with aggregated Base Model parameter values using our parameter morphism method (Sections 3 and 6). While primarily impacting latency, it may also influence connection probability in a more elaborated model.

```
external event with Flare
<%
// LatencyDegredationFactor comes from morphism
System.out.println("Incident solar flare at time t = " + currentTime);
MeanLatency = lookup table value for increased noise environment

TransitionInfo ti;
ti = ctm.getTransitionInfoFor("tryToConnect", "timeToHandshake");
TimeInfo timeInfo = ti.getTInfo();
timeInfo.setMean(MeanLatency);
ti.setTInfo(timeInfo);
%>!
```

Figure 6: DEVS event handling of a signal flare event.

4 Task 2: Algorithm Development

This section contains a summary of the work necessary to complete this task. Appendix A and Appendix B provide background details on the mathematical concepts that underpin the methods and tools used to complete this task. Appendix C provides details on the ontology developed for this project. Appendix D provides detail on the graph databases used for this project. Appendix E provides insights to future potential and likelihood for the direction of this project in future phases.

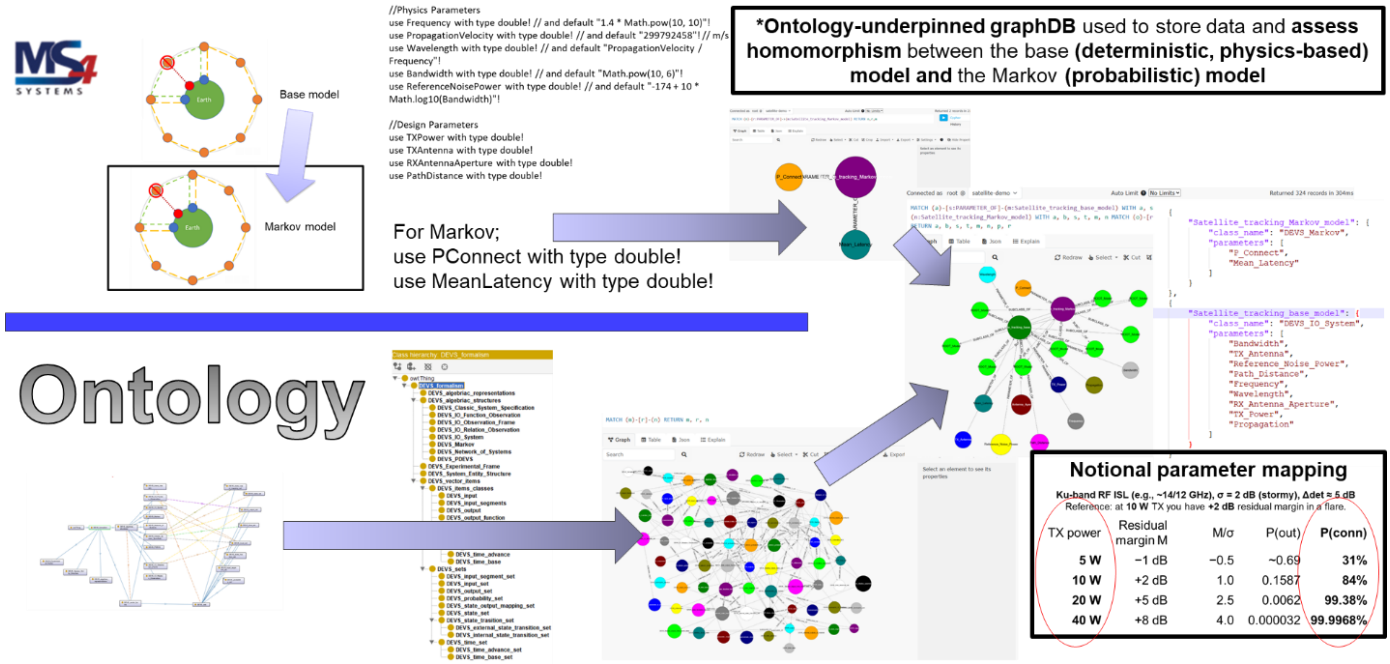


Figure 7: Illustration of the convergence of modeling & simulation, ontology, and graph graph-based algorithmic analysis

We completed a version of the algorithms used to express relationships between physics-based, deterministic models (referred to the as the base model) and the probabilistic, Markov models (referred to as the lumped model). To achieve completion of the task, we (1) completed a version of the DEVS-based ontology in Protégé (See Appendix C), (2) converted the ontology to data structures to underpin the graph databases (graphDBs) (See Appendix D), and (3) ran native graphDB analytics to demonstrate algorithmic determination of the homomorphic equivalence between the base and lumped models. Illustrated in Figure 7, the ontology enables harmonization of the data from the two models in MS4 Systems and the graphDBs. A notional example of data harmonization is the automated translation of TX power to the probability of connection between satellites. End-to-end data/model harmonization is expected as part of the future phases of this project.

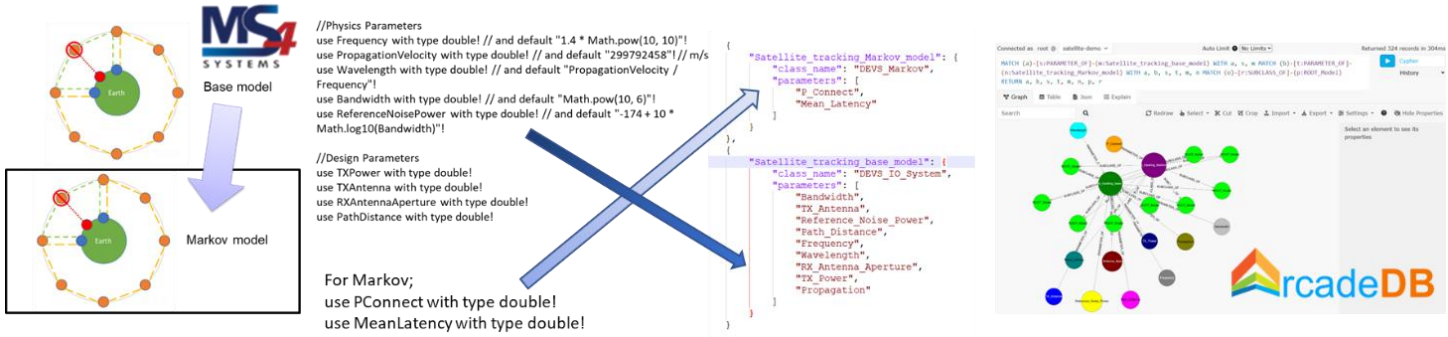


Figure 8: JSON format representation of the models in MS4 Systems (left) and ArcadeDB (right)

The current version requires manual exchange between MS4 Systems and the graphDBs. However, conversion of the ontology to underpin the graphDBs has an established workflow. The algorithms to check for homomorphism between models and compute values also have established workflows. In future phases, the models will be linked such that if a probability of connection or mean latency is establish as a key performance parameter (KPP), the flow-down to the physics of the design will be automated. Thus, working in reverse, changes to the design can be automatically connected to performance against KPPs. For example, as suggested in the bottom right of Figure 7, the transmitter (TX) power may be automatically connected to the probability of connection between satellite nodes, and vice versa. Additionally, Figure 7 illustrates the parameter and model mapping between MS4 Systems and ArcadeDB showing the flow of parameter information from the Base model to Lumped model. In brief, in MS4 Me (RTSync's simulation tool) parameters such as Frequency are declared in the form: "Use Frequency with type double …". When assigned values, these parameters are used in subsequent model execution. These parameters can be accessed by the ArcadeDB through the JSON representation in the middle of Figure 8. As a future goal, an expression in the JSON representation has been written by the ArcadeDB (and Neo4j) that, when executed by MS4 Systems, will assign the values required to drive the simulation, thus contributing to the future harmonization of the ecosystem.

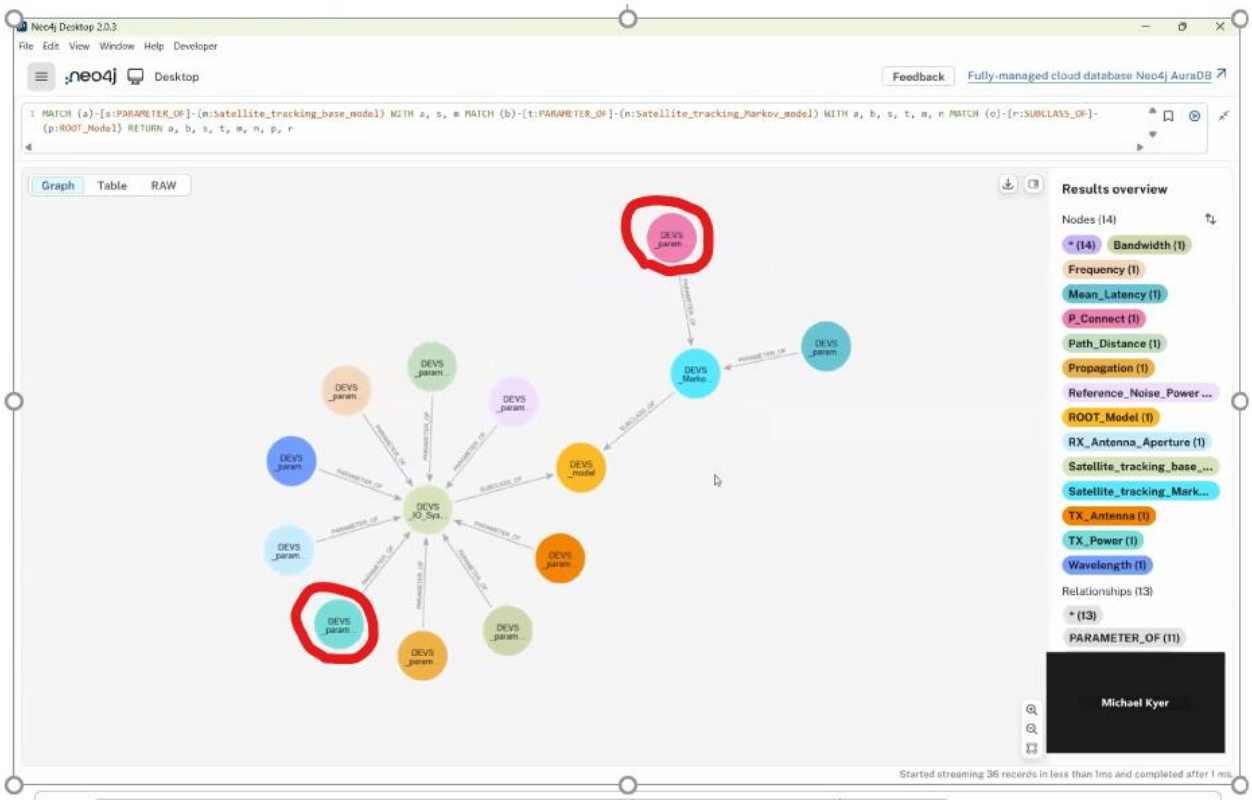


Figure 9: Snapshot from the graphDB Neo4j showing the algorithm in use to capture the homomorphism between the physic-based, deterministic model (~bottom left) and the probabilistic, Markov model (~top right). The red outline of the teal circle and pink circle are used to illustrate the parameter mapping, which is not shown in this image.

To provide further detail, Figure 9 shows a snippet from Neo4j after an algorithmic query that displays the homomorphic relationship between the Base and Markov models. There are two clusters. The larger cluster with a light green circle at the center (lower left of Figure 9) represents the physics-based model. The small cluster with light blue and darker green and pink circle branching (upper right of Figure 9) represents the Lumped model. The yellow-ish, orange circle between the clusters (at the middle of Figure 9) represents the existence of a homomorphism between the two models. What is not shown in Figure 9 is the parameter

mapping between the models. In Figure 4, we show the mapping of TX power and the probability of connection. In Figure 9, this is represented by the red outline around the graph nodes that represent TX power (~teal at the bottom left) and probability of connection (pink at the top right). The connection between the two models is shown as the "subclass" relation but will be suitably characterized in the next work, for example as "has_morphism_to." The final demonstration illustrates the detailed mapping between parameter as well as the interconnection between the graphDBs and MS4 Systems. For further details, refer to Appendix D.

For the Feasibility Demonstration we have selected to focus on the use case of a solar flare causing outage/latency in transport, which we have instantiated into the algorithm thread.

5 Task 3: Test Environment Development

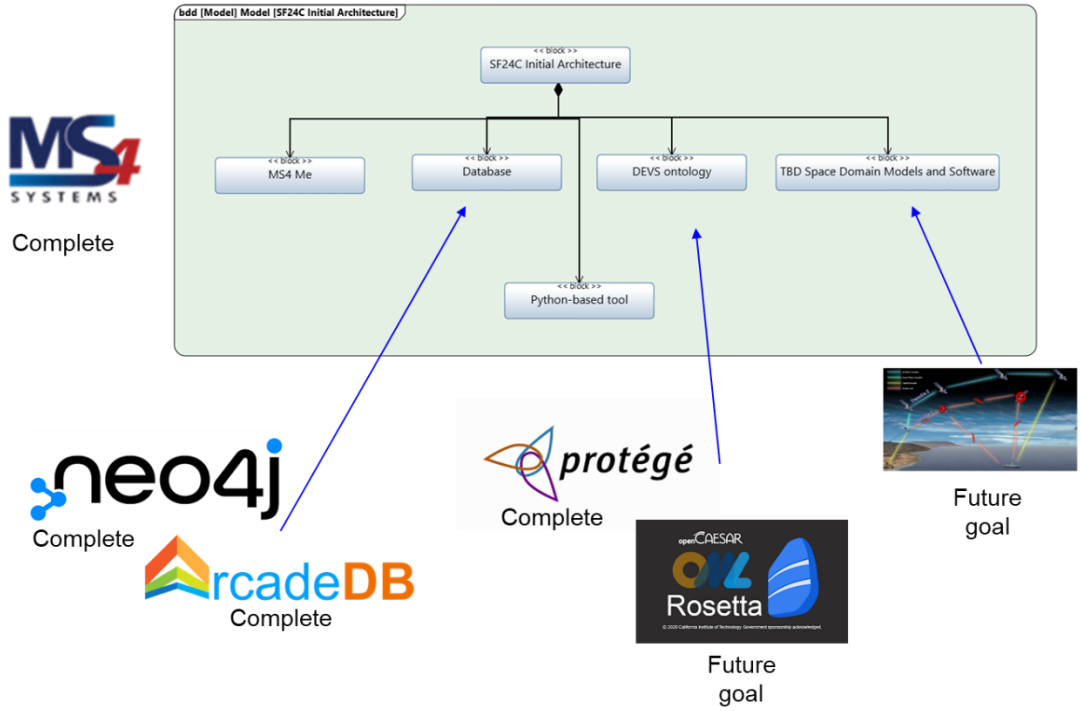


Figure 10: Test Environment

We created the generic architecture model shown in the light green, rectangular shape of Figure 10. We selected and developed the specific components of the test environment. While our initial decision favored ArcadeDB as the preferred database, we subsequently evaluated the benefits of Neo4j and concluded that incorporating both databases would incur minimal additional cost. Therefore, we have completed development of both ArcadeDB and Neo4j. Both are graphDBs, which enable storage of the models as knowledge graphs and the use of graph-based algorithms to complete the desired analysis. For the ontology, we have stayed with Protégé, which is sufficient for this phase. Moreover, it was straightforward to transfer the ontology to form the data structure for the graphDBs. As a stretch goal, we are explored use of Rosetta for future ontology modeling and analysis. Protégé is a visual ontology modeling tool and Rosetta is a textual-code ontology modeling tool. The advantage to using Rosetta is to treat the ontology as code, which enables use of common software development practices such as Git repositories, among other software analysis capabilities. Also, as a stretch goal is the addition of space domain software, such as the VT-NSI SpaceNet (Distributed Space and Terrestrial Networking Infrastructure for Multi-Constellation

Coexistence) software. This is advantageous because SpaceNet is focused on modeling networks of transport satellites, which is the use case focus for this phase of the project. However, addition of this type of software was not critical to the outcome of this phase and is therefore listed as a stretch goal. Therefore, we selected not to include Rosetta and SpaceNet for this phase and expect their inclusion in future phases of the project.

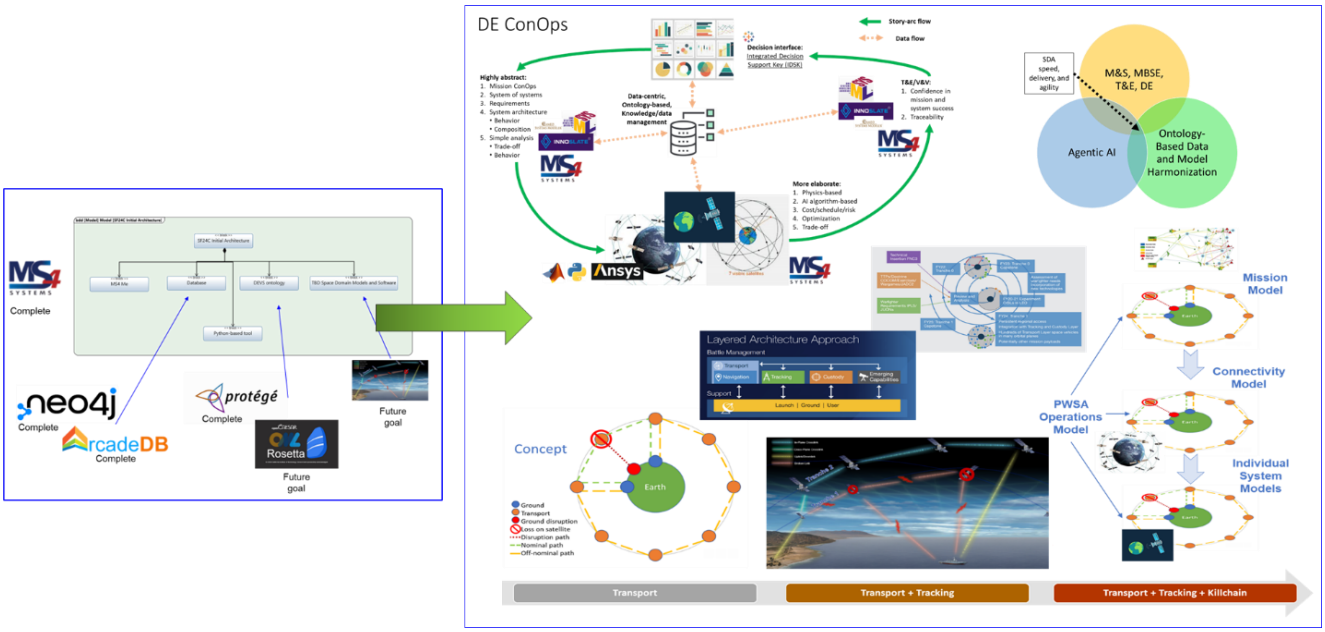


Figure 11: Illustration of the future vision and transformation expected from the current, small-scale test environment to a much larger scale of mission scope, digital engineering flow, and intersect with advanced methods such as agentic AI

Besides demonstration of capability, an outcome for this phase is understanding of plans for future phases. It is desirable to consider typical engineering artifacts (e.g., CAD models) and agentic AI that enable human-machine teaming for enhanced efficiency and effectiveness. Therefore, we investigated such considerations for scaling the products to that of the SDA context and overarching DoD digital ecosystems (such as is envisioned in Figure 11). In this vision, we anticipate growth from solely focusing on the transport layer to transport plus tracking plus kill chain, and on to acquisition (not shown in Figure 11). We envision a digital engineering (DE) concept of operations (ConOps) similar to the loop shown in the top left of the larger box in Figure 11, in which data flows harmoniously through the ecosystem and to/from decision maker interfaces. We envision agentic AI as playing a pivotal role in the future state, which will intersect with other common DE enablers such as M&S, MBSE, and T&E. For more detail on our vision for agentic AI, refer to Appendix E.

6 Task 4: Feasibility Demonstration

As part of our overall feasibility demonstration, we developed a conceptual experimental frame for a solar flare event impacting satellites along the nominal transport path from one ground station to the next, see Figure 12. Solar flares and associated solar radio bursts can introduce significant variability in the near-Earth electromagnetic environment, particularly at lower radio frequencies. These events deposit large amounts of energy into the upper atmosphere, increasing ionization in the ionosphere and thermosphere. The resulting “swelling” of the upper atmosphere can amplify scintillation effects and substantially increase atmospheric drag on satellites in low Earth orbit. Naturally, these atmospheric disturbances also impact communication channels, with HF systems experiencing severe attenuation and even complete radio blackouts during major flares, while higher-frequency systems in the L- and S-bands can still exhibit

temporary reductions in carrier-to-noise ratio. However, for intersatellite links that typically transmit at higher frequencies and do not propagate through the atmosphere or ionosphere, these channel effects are significantly diminished, especially for constellations that are in higher altitudes (i.e., greater than 400km).

While the overall risk to intersatellite communication links is low, solar radio and microwave emissions at frequencies coinciding with those used by spacecraft can act as a form of natural jamming, but only under specific geometric alignments where the Sun lies within the antenna's main beam or strong sidelobes.

The solar irradiance in the S-bands, for example, (i.e., at approximately 3 GHz) has been shown to vary significantly in the presence of increased solar activity. During major solar events the flux can reach 1,000–10,000 SFU at 3 GHz, leading to a solar flux density of 10^{-19} to 10^{-18} $[Wm^{-2}Hz^{-1}]$.

Most solar radio bursts are short duration (i.e., seconds to approximately 15 minutes), broadband, and incoherent emissions. Therefore, they can be modeled as a noise spectral density, rather than a coherent interference signal. Thus, solar flares and radio bursts can be easily represented as an increase in system noise temperature that scales with solar flux intensity. This in turn can be mapped to an environmental loss parameter within the link budget, capturing the transient increase in system noise temperature associated with elevated solar flux. This approach enables efficient assessment of solar-event impacts on intersatellite communication performance without overstating the operational risk.

Given this, a scenario consists of a communication message generated at t=0 which starts off the message flow. At t=1, the solar flare event occurs impacting a satellite along the nominal path. As the solar intensity increases, the SNR of the system decreases, leading to an increase in packet error rate and thus an increase in expected retransmissions. This will either make it too difficult to establish a connection or adds significant delay to the latency.

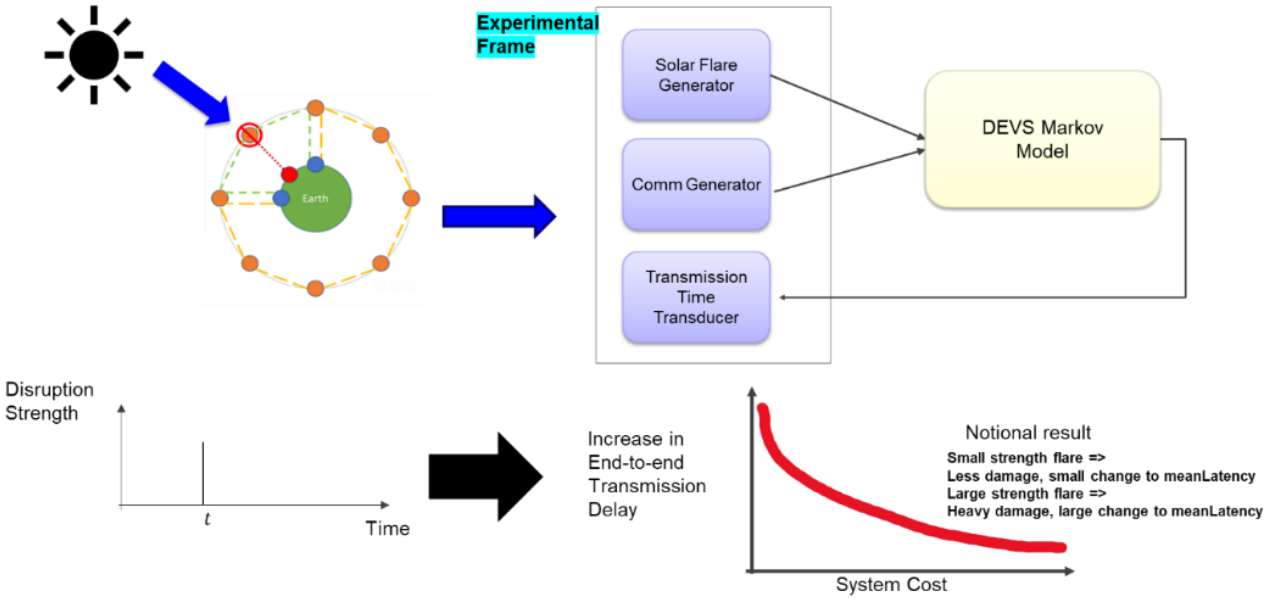


Figure 12: Solar flare use-case demo for testing our framework.

For initial representation, we abstract the system cost as aggregated from engineering design aspects to show a degradation in transmission time. The realization of this model in MS4 Me is shown in Figure 13 where each of the small boxes represent DEVS atomic models. The arrows between them indicate the port-to-port coupling which follows the message flow.

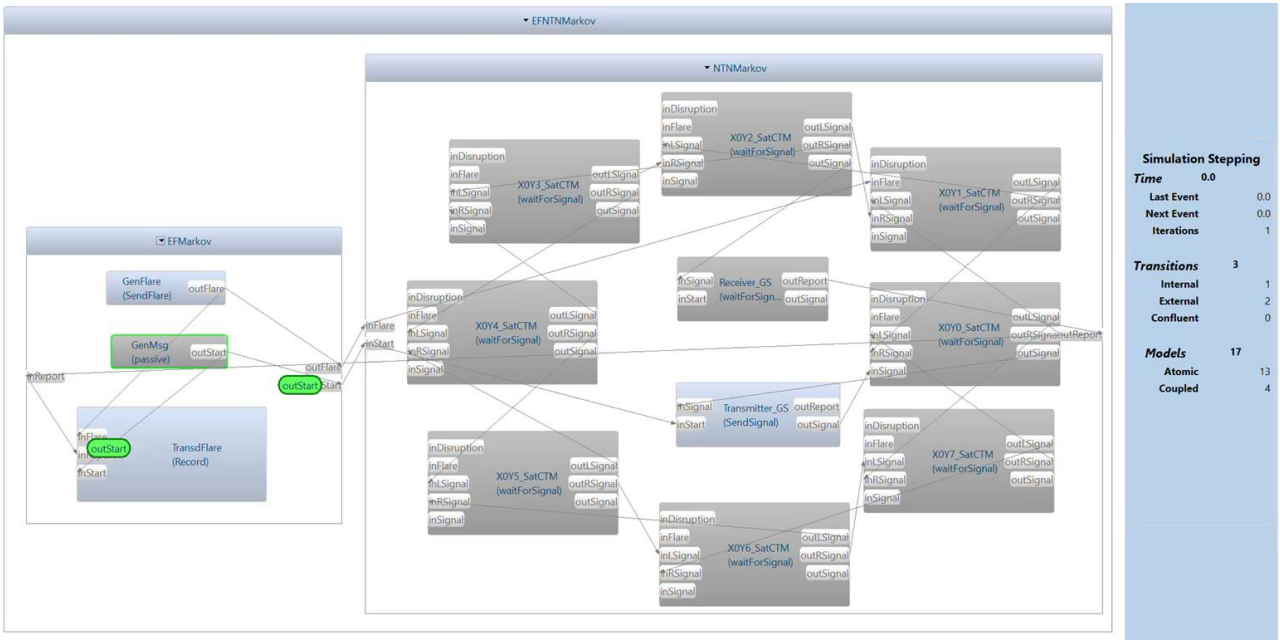


Figure 13: MS4 Me realization of DEVS NTN model.

Applying this use-case experimental frame to the DEVS Markov model leads to a set of statistics for the arrival time of the transmission message. In this test, we consider the probability of connection = 1.0 and solely look at the comparison of end-to-end transmission time. The single hop transmission latency between satellites is mapped from varying degrees of transmitter power. Power consumed by single-hop transmissions can be loosely related to the cost of the system. For each parameter setting, we execute 1000 Monte Carlo samples to produce the plot in Figure 14. The orange line is the case that no flare occurs (as a base line performance of the system at each cost). Then the blue line is for the solar flare data. The takeaway from a result like this is that there is a greater disparity between average end-to-end time for low-cost systems. In the table on the right, we summarize a comparison of 3 scenarios. In a nominal space environment, the noise is relatively low only requiring a standard amount of power to transmit messages with a low latency. During a solar flare event the environmental noise increases dramatically where nominal power output leads to increased latency. To mitigate this, a relatively small increase in power can restore nominal latency despite the high noise environment.

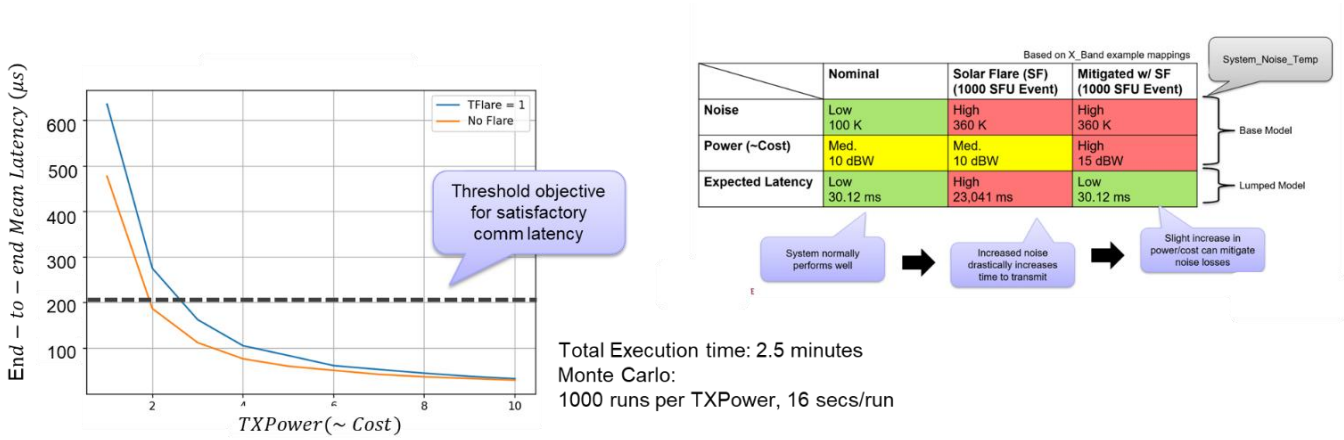


Figure 14: Results of Monte Carlo simulations.

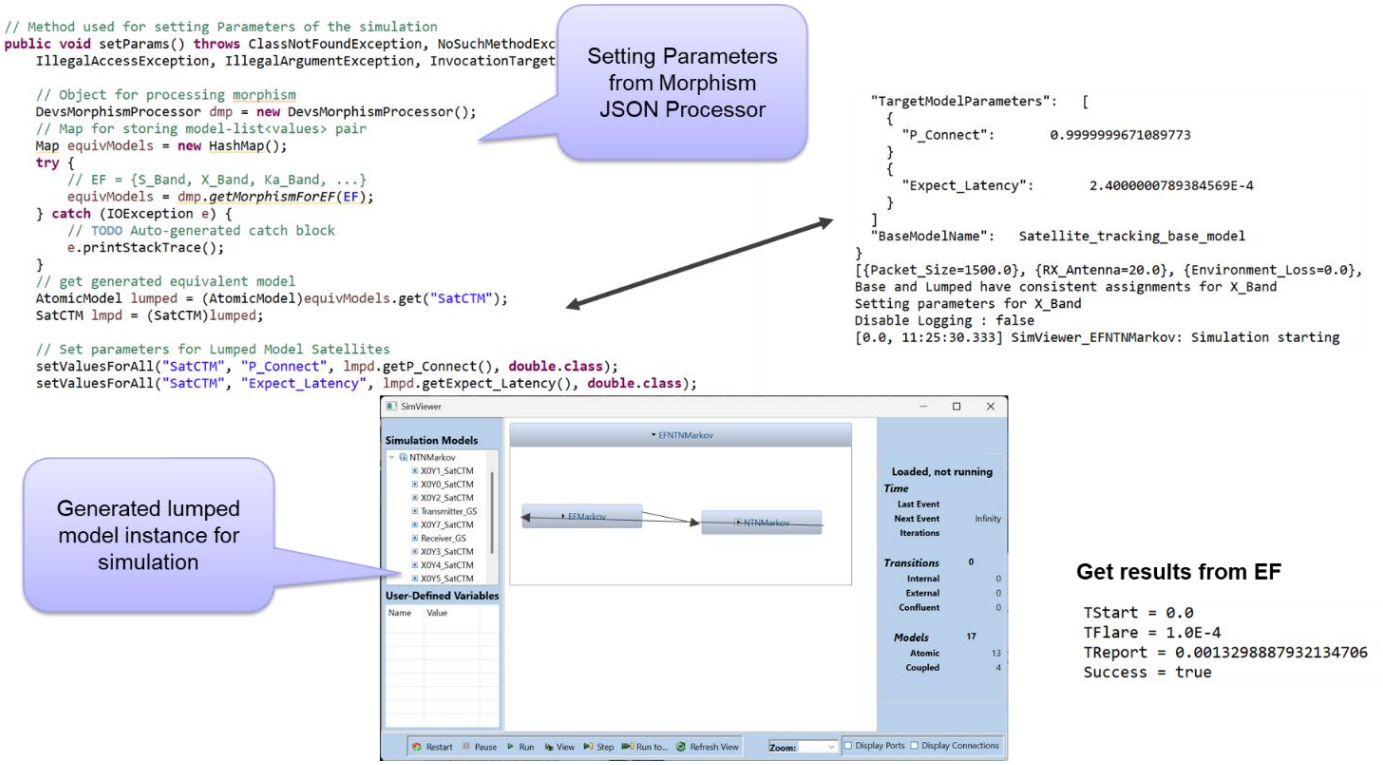


Figure 15: Summary of morphism process on the simulation side in MS4 Me.

Figure 15 illustrates the execution of the morphism applier software within the simulation environment, where parameter values mapped from the ontology are ingested in JSON format. As shown at the top of Figure 15, operating within a run-control framework, the system applies values derived from a single-hop analysis to instantiate the lumped model, referred to here as "SatCTM." The script demonstrates how scenario settings from the X-band base model are utilized to generate a corresponding simulation of the lumped Markov model within the simulation interface. This setup enables execution of individual Monte Carlo samples for performance evaluation illustrated by the simulation viewer at the bottom of Figure 15.

6.1 Morphism Modeling Application

The morphism modeling application developed by the team is a computational tool designed to identify and exploit structural morphisms, such as homomorphisms, between two satellite constellation system models. Its primary goal is to establish a formal mapping between different model representations—whether they vary in fidelity, configuration, or environmental conditions—while preserving the essential behavioral and performance characteristics of the system. By leveraging ontology-driven graph representations, the application provides a structured and semantically rich framework for model comparison, transformation, and performance estimation across diverse modeling contexts.

The application connects a domain-specific ontology to graph databases to support efficient model representation, querying, and morphism discovery. Through this integration, users can populate and interact with databases such as Neo4j and ArcadeDB, which serve as the backbone for storing and visualizing model relationships. The ontology ensures that all entities and attributes are consistently defined across systems, allowing the database to function as an intelligent repository of model components and parameters. Users can retrieve comprehensive parameter lists for individual models, assign or modify parameter values, and identify all models sharing specific characteristics. This integration with existing graph database infrastructures also enables the use of powerful native visualization and querying tools, making it easier to explore model relationships and dependencies.

A central capability of the application is its ability to detect whether an existing or definable morphism exists between two models. When an existing morphism is identified, the application directly maps attributes from the base model to the target model, ensuring an accurate translation of key physical and system parameters. These include constants such as the speed of light and Boltzmann's constant, as well as operational parameters like frequency, bandwidth, antenna characteristics, system noise temperature, environmental losses, and spectral efficiency. Using these inputs, the application computes a variety of critical performance metrics, including wavelength, free-space path loss (FSPL), aggregate losses, received power, signal-to-noise ratio (SNR), bit rate, energy-per-bit-to-noise ratio (Eb/N_0), bit error rate (BER), packet error rate (PER), success rate, and transmit latency. These calculations allow users to assess how well a given configuration supports communication between satellites under defined conditions.

In cases where an existing morphism cannot be established, the application searches for a definable morphism by examining the structural proximity of models within the graph database. The underlying assumption is that models located near one another in graph space exhibit similar characteristics. Missing attributes in the target model are inferred from those of the closest related model, enabling the creation of a definable mapping. *Once the missing data are supplemented, the same set of calculations is performed, though the results are interpreted with awareness of the potential uncertainty introduced by the approximations. This capability allows the system to operate effectively even in situations where incomplete or imperfect model data are available, supporting flexible and resilient analysis workflows.* We note that this is an instance where the implementation of morphism tools to augment those of the ontology provides an important capability to support SDA system development over the long term. Our proposal for further development includes enhancing the initial tool development performed in this phase.

All results are automatically exported in a structured, machine-readable JSON format that includes the attributes and computed values for both the base and target models, along with the morphism type (existing or) and the corresponding mapping details. This output format is designed for easy integration into external analysis tools, simulation environments, or data visualization pipelines, promoting interoperability across modeling frameworks and disciplines.

Beyond simple fidelity transformations, the morphism application could potentially be applied to a wide range of scenarios. For example, in the feasibility demonstration, we showed how morphism support simulation of model transitions between systems operating under nominal versus degraded conditions, (in particular, when a satellite communication link experiences increased noise due to solar flare activity or other space weather phenomena.) It could also be used to evaluate the impact of changing network configurations, orbital geometries, or antenna alignments on system latency and connectivity between satellites. In each case, the application provides a rigorous and automated means of quantifying how modifications to one model—or external environmental factors—affect the performance and reliability of another. Through this capability, the tool serves as a bridge between models of differing structures or conditions, ultimately enhancing the understanding and optimization of satellite communication networks.

7 Future Work Planned

The overall objective of a next phase will be to enhance the validation of the approach and demonstrate its applicability with increased realism of the scenario. The recommended tasks are as follows:

First, the team must expand the use case. The current focus has been constrained to the transport layer of the PWSA, which also includes a tracking layer. Therefore, the next phase of the project will focus on addition of the tracking layer. Ideally, the project should also consider inclusion of the kill chain, which enables the full mission operation from tracking to transport to kill chain. However, this may be deferred to a follow-on project phase. Additionally, elaboration of the current physics-based model to support cost-informed mitigation strategies could be done to increase system and mission resilience following a disruption. Furthermore, the complexity of the models will increase from simple, 2D models to

multidimensional networks, which is more realistic in the SDA application context. Figure 16 is used to articulate the expectations for the next phase of the project in terms of expanding the use case.

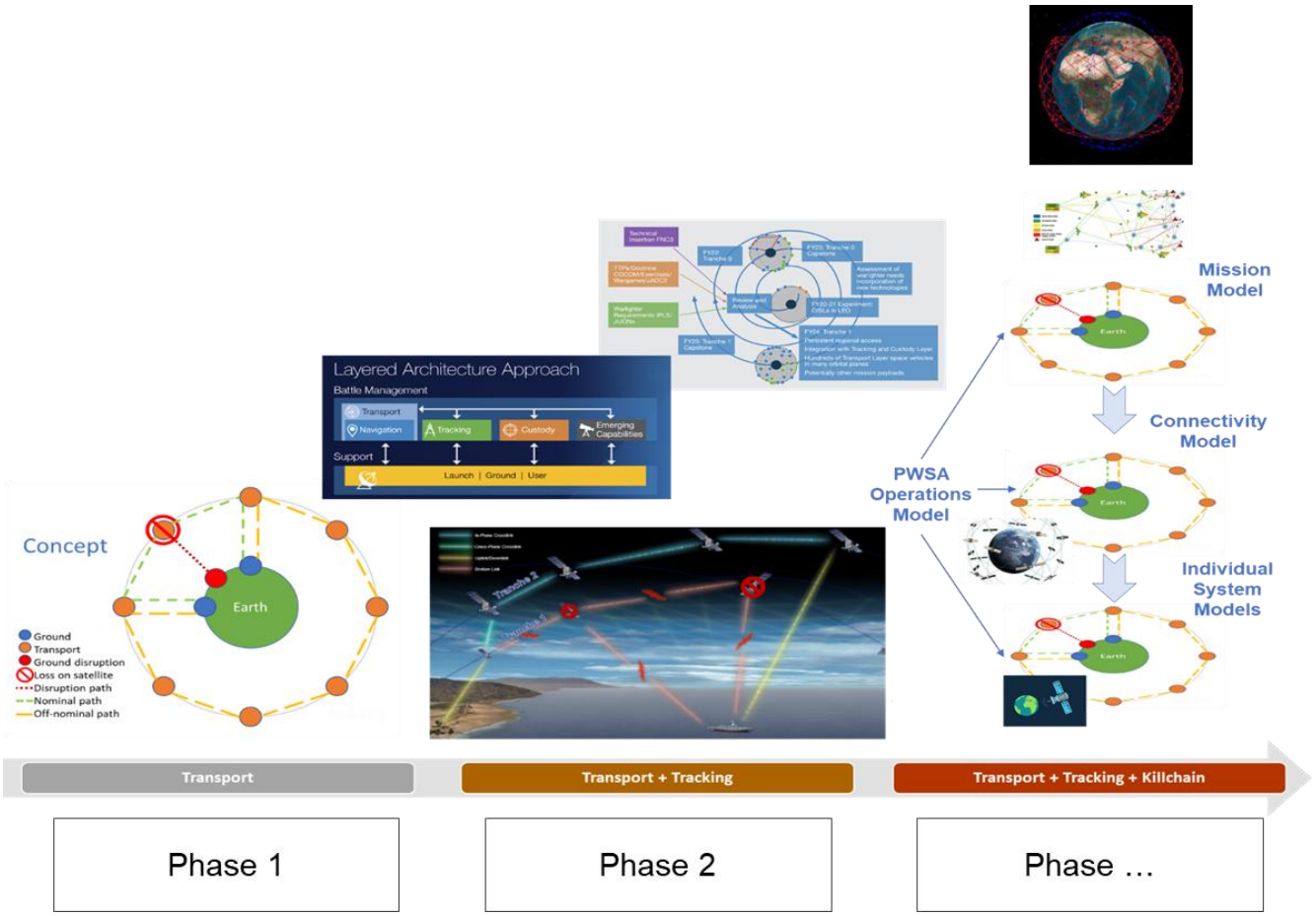


Figure 16: Next phase of the project in terms of expanding the use case.

Second, we will expand the software underpinning the developed tools and methods. For comparison, in the current phase of the project, the infrastructure consists of the (1) DEVS-PWSA framework, (2) algorithms, and (3) overarching test environment. The DEVS-PWSA framework is expected to add other elements of the PWSA (e.g., transport models). The algorithms will add additional capabilities such as new algebraic structures and enhanced mapping of such structures. This will include enhancement of the morphism modeling application tool to support discovery of definable morphisms and the evaluation of such morphisms for their accuracy in deriving lower resolution model parameter values from the source higher resolution parameter settings. The next phase should also streamline the workflows. The current process is largely manual (e.g., the exchange of JSON files betweenMS4 Me and graphDB). Such processes should be streamlined to increase the practicality for insertion into SDA development processes. Additionally, the architecture should be expanded to be closer to the realism shown previously in Figure 11. This expansion could include other software such as Cameo, Matlab, Ansys STK, Innoslate, and Rosetta as well as current STTR/SBIR tools such as TensorX and Sedaro. Furthermore, future phases should include agentic AI in which an orchestrator model acts as part of the user interface as well as tasks, reviews, and synthesizes results from other tools within the digital ecosystem. Figure 17 is provided as an illustration of the notional agentic AI framework and architecture that is envisioned for the next phase of the project.

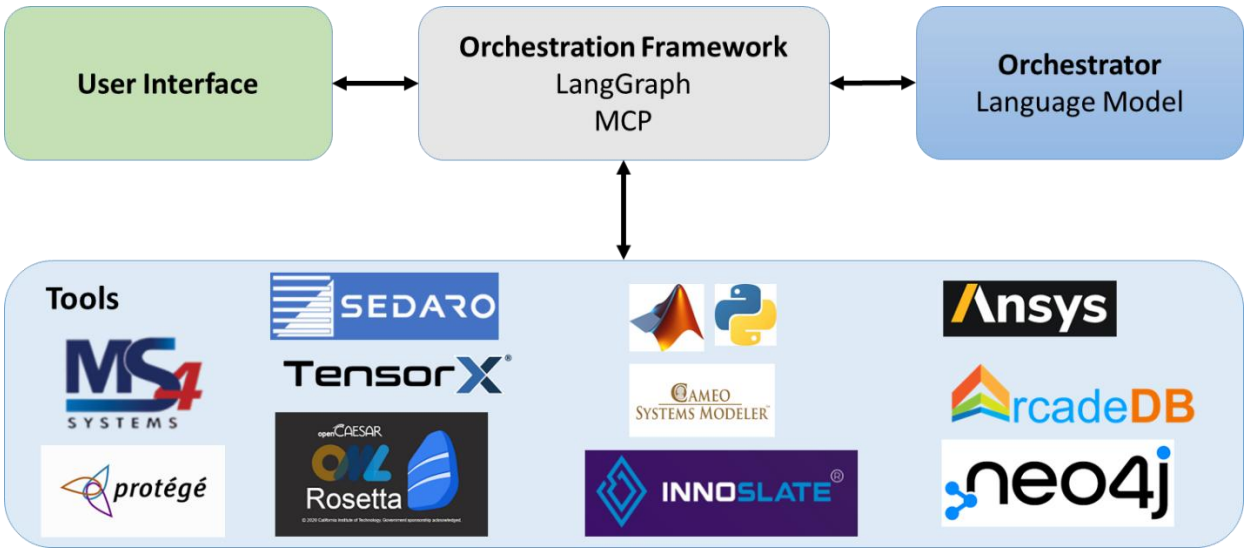


Figure 17: Notional agentic AI framework and architecture

Appendix F presents the slide deck presented at the final presentation held on October 30th, 2025.

References

1. Abdurrahman A I, Sarjoughian. H., Model-Driven Time-Accurate DEVS-Based Approaches for Design, TMS/DEVS, SpringSim, 2018
2. Agrawal R. and C. Fernengel, The Kill Chain In Space: Developing A Warfighting Mindset, Texas National Security Review, October 24, 2019
3. Aliyu, T, Olanrewaju, Oumar Maïga, Mamadou Kaba Traoré. 2016. The high level language for system specification: A model-driven approach to systems engineering, February 2016
4. Amissah, T, Handley, Seck, "Towards a Framework for Executable Systems Modeling: An Executable Systems Modeling Language (ESysML), SpringSim 2018
5. Bocciarelli, A , Giglio, Paglia, "Model Transformation Services for MSaaS Platforms" SpringSim 2018
6. Ceylan, Bulut Ozan, Emre Akyuz, and Ozcan Arslan. "Systems-Theoretic Accident Model and Processes (STAMP) approach to analyse socio-technical systems of ship allision in narrow waters." Ocean Engineering 239 (2021): 109804.
7. Davis, P. Broad and Selectively Deep: An MRMPM Paradigm for Supporting Analysis. Information 2023, 14(2), 134; https://doi.org/10.3390/info14020134.
8. De Saqui-Sannes, P., Vingerhoeds, R. A., Garion, C., & Thirioux, X. (2022). A taxonomy of MBSE approaches by languages, tools and methods. *IEEE Access*, *10*, 120936-120950.
9. Frasier, Douglas M., Gorenc, Frank, and Shapland, John S., Hypersonic Defense Requires Getting Space Sensor Right, Real Clear Defense, May 13, 2020
10. Freeman, L. et al., Positioning Test and Evaluation for the Digital Paradigm (2023). The Journal of Test & Evaluation, Volume 44, Issue 2.
11. Hall, Steven B, Zeigler, Bernard P., and Sarjoughian, Hessam, Joint MEASURE: Distributed simulation issues in a mission effectiveness analytic simulator, 1999, https://api.semanticscholar.org/CorpusID:59629869}
12. Harkleroad, E., A. Vela, and J. Kuchar. "Review of systems-theoretic process analysis (STPA) method and results to support NextGen concept assessment and validation." Project Report: ATC-427, MIT, Lincoln Laboratory, Washington, DC, Oct 25 (2013): 2018-12.
13. Imbert I, Cecilia Zanni-Merk and Lina F Soualmia Combining DEVS simulation and ontological modeling for hierarchical analysis of the SARS-CoV-2 replication ,Simulation: Transactions of the Society for Modeling and Simulation International 2023, Vol. 99(10) 1011–1039
14. Kewley, R., Kester, N., & McDonnonell, J. DEVS Distributed Modeling Framework: A Parallel DEVS Implementation via Microservices. Proceedings of SpringSim, 2016
15. Lammers, C. J. Steinman, M. Valinski, K. Roth. "Five-Dimensional Simulation for Advanced Decision Making," SPIE—Enabling Technologies for Simulation Science XIII, Paper SPIE 734
16. Larson, W. J., et al. (2009). Applied Space Systems Engineering, The McGraw-Hill Companies, Inc.
17. Laser Communications:Space Development Agency Should Create Links Between Development Phases, GAO-25-106838 Published: Feb 26, 2025. Publicly Released: Feb 26, 2025. https://www.gao.gov/products/gao-25-106838(U)
18. Lee C, Mitchell institute, https://michellaerospacepower.org/wpcontent/uploads/2023/06/bolsterig-arctic-domain-awareness-final.pdf
19. Marchau, V.A.W.J.; Walker, W.E.; Bloemen, P.J.T.M.; Popper, S.W. Decision Making under Deep Uncertainty: From Theory to Practice; Springer Nature: Cham, Switzerland, 2019.

20. MS4 Me, http://www.ms4systems.com/pages/ms4me.php
21. Petty, M D., Eric W. Weisel Model Composition and Reuse, in: Model Engineering for Simulation, Eds: L. Zhang et al. Elsevier, 2019
22. Proliferated Warfighter Space Architecture (PWSA) Link-16 Mission Thread for USINDOPACOM (url: https://www.sda.mil/wp-content/uploads/2024/05/Transport-Layer_distro-A_FINAL.pdf)
23. Rice, Roy E. Calculating the Probability of Successfully Executing the Kill Chain to Analyze Hypersonics Phalanx, Spring 2022, Vol. 55, No. 1 (Spring 2022), pp. 22-27
24. Salado, A. and P. Wach (2020). Interpretation Discrepancies of SysML State Machine: An Initial Investigation. 18th Annual Conference on Systems Engineering Research (CSER) 2020. Redondo Beach, CA.
25. Sanders C.. A Call for a New Army Modeling Framework, Conference: Simulation Innovation Workshop Winter, Orlando, FL, February 2018
26. Santucci, JF, Laurent Capocchi1 and Bernard P Zeigler, System entity structure extension to integrate abstraction hierarchies and time granularity into DEVS modeling and simulation, Transactions of the Society for Modeling and Simulation International, 2016, Vol. 92(8) 747–769
27. Seo, C. B.P. Zeigler, and DH Kim, DEVS Markov Modeling and Simulation: Formal Definition and Implementation, SpringSim, Baltimore, MD 2018
28. Tao, Hoong Yan See, et al. "Challenges and Opportunities in the Digital Engineering Simulation Curriculum Development." 2023 IEEE International Systems Conference (SysCon). IEEE, 2023.
29. Tolk, A. Simulation-Based Optimization: Implications of Complex Adaptive Systems and Deep Uncertainty. Information 2022, 13(10), 469;
30. USDOD (2020). DOD Instruction 5000.89 Test and Evaluation. OUSD(R&E) and DOT&E.
31. Wach, P., & Salado, A. (2022). The need for semantic extension of SysML to model the problem space. In *Recent Trends and Advances in Model Based Systems Engineering*(pp. 279-289). Cham: Springer International Publishing.
32. Wach, P., Beling, P., & Salado, A. (2023). Formalizing the Representativeness of Verification Models using Morphisms. *INSIGHT*, *26*(1), 27-32.
33. Wach, P., Beling, P., Zeigler, B. P., & Salado, A. (2023, March). Study of equivalence in systems engineering within the frame of verification. In Conference on Systems Engineering Research (pp. 197-210). Cham: Springer Nature Switzerland.
34. Wach, P., et al. (2021). "Conjoining Wymore's Systems Theoretic Framework and the DEVS Modeling Formalism: Toward Scientific Foundations for MBSE." Applied Sciences **11**(11): 4936.
35. Wach, P., et al. (2022). "Pairing Bayesian Methods and Systems Theory to Enable Test and Evaluation of Learning-Based Systems." INSIGHT **25**(4): 65-70.
36. Winfrey, C. M., Baldwin, B. A, Cummings, M. A, & Ghosh, P. OSM – An Evolutionary System of Systems Framework for Modeling and Simulation, Proceedings of the 2014 Annual Simulation Symposium, 2014
37. Wymore, A. W. (1993). Model-Based Systems Engineering. 2000 NW Corporate Blvd., Boca Raton, FL, USA 33431, CRC Press LLC.
38. Zeigler, B. P. The System Entity Structure: Knowledge Representation for Simulation Modeling and Design," (with G. Zhang) in: *Artificial Intelligence, Simulation and Modeling*, (Eds. L.A. Widman, K.A. Loparo, and N. Nielsen), J. Wiley, NY, pp. 47-73, 1989.
39. Zeigler, B. P., Muzy, A., and Kofman, E., Theory of Modeling and Simulation (3rd ed.), Academic Press, Elsevier 2018

40. Zeigler, B. P., Seo, C., and Kim, D. “DEVS Modeling and Simulation Methodology with MS4 Me Software”. Spring Sim 2013 San Diego, CA, USA.
41. Zeigler, B. P., Simulation-based Evaluation of Morphisms for Model Library Organization, in *Model Engineering for Simulation,* Eds: Lin Zhang, Y. Luli, and B. P. Zeigler, Model Engineering for Simulation, Elsevier 2019
42. Zeigler, B.P. (2022) ‘A methodology to characterize simulation models for discovery and composition: a system theory-based approach to model curation for integration and reuse’, Int. J. Simulation and Process Modelling, Vol. 19, Nos. 1/2, pp.3–13.
43. Zeigler, B.P. and Y. Moon. Abstraction Methodology Based on Parameter Morphisms, Proc. Enabling Technology for Simulation Science, SPIE AeoroSense 97. 1997. Orlando, FL.
44. Zeigler, Bernard P., Seo, Chungman, and Kim, Doohwan, “System Entity Structures for Suites of Simulation Models,” *Int. J. Model., Sim., Sci. Comp.,* vol. 4, no. 3, September 2013.
45. Zeigler, BP. DEVS and MBSE: A Review, International Journal of Modeling, Simulation, and Scientific Computing VOL. 13, NO. 02 (2022)
46. Zeigler, B. P., Kim, D., Keller, N., Ceney, J., Supporting the Reuse of Algorithmic Simulation Models, SummerSim, July 2020

Appendix A: Background on MBSE and M&S

Current practices followed by SDA tend to rely on document-based practices that are static and siloed, and impede the desired speed, delivery, and agility. As indicated above, to deliver capabilities and systems to the desired speed and with the desired agility, SDA must look to data-centric approaches that are transformative in nature.

The defense industry began its digital transformation in earnest after the release of the digital engineering strategy in 2018. Although digital methodologies have been around for decades, true transformation is still emerging. Even within the same set of tools, different organizations often have varied implementations, leading to a lack of reuse and interoperability. This challenge is particularly pronounced in mission engineering, where interdependent organizations must collaborate to develop, operate, and maintain systems and capabilities suitable for warfighters. For instance, models created in model-based systems engineering languages and tools, such as Systems Modeling Language (SysML) and Cameo, often adhere to a "style guide" that is organization-specific [4]. Overcoming these obstacles is a crucial step in achieving reuse and interoperability, which are essential for speed and agility.

As the complexity of systems and capabilities grows, so does the complexity of the models representing them. Are these models intended for human analysis or for computer analysis that is then interpreted for humans? Human consumption of complex models is challenging. Even in less complex scenarios, SysML and Cameo models can be interpreted in various ways [5]. Additionally, MBSE lacks a common underlying mathematical theory, unlike computational fluid dynamics and finite element analysis. While SysMLv2 has ontological underpinnings, it lacks a mathematical foundation. To manage the complexity of missions, capabilities, systems, and models, SDA should seek a digital framework supported by a robust mathematical theory.

True digital transformation involves making effective and efficient decisions and communication by connecting the right data correctly. Two mathematical frameworks have been proposed for integration to support this transformation [6]: the Tricotyledon Theory of System Design (T3SD) [7], known for coining the term MBSE, and the Theory of Modeling & Simulation (commonly referred to as Discrete Event System Specification, or **DEVS**) [6]. The advancement of MBSE, grounded in T3SD, is gaining momentum, while DEVS is widely recognized in the modeling and simulation community, with over 10,000 publications. Our project aims to apply research towards integrating T3SD and DEVS in the space domain, specifically to achieve digital test and evaluation (dT&E) for the SDA.

DEVS background

The DEVS framework stands out for its hierarchical and modular architecture. Within this structure, a system or System-of-Systems (SoS) model consists of several individual models, each representing a system component. These models are classified as either atomic (indivisible) or coupled (comprising other models). DEVS distinctly treats agent-based and other simulation elements as specific atomic or coupled models, complete with their state transitions, inputs, and outputs. By adopting a systems-theory approach, DEVS ensures the integrity of each simulator's fidelity, be it high-resolution or a more generalized representation, while fostering their collaboration. The versatility of the DEVS framework is further emphasized by its support for varied M&S paradigms [8]. Despite the potential differences in atomic or coupled models—like differing state variables or timing mechanisms—the overarching DEVS simulation protocol ensures their correct interaction. This adaptability not only allows DEVS to model and simulate a diverse range of systems but also facilitates the amalgamation of independently crafted models, promoting efficiency and reusability.

Agent-based modeling is among the paradigms specifically supported by RTSync's DEVS Markov modeling capabilities [9]. DEVS Markov models can be individualized with specific transition probabilities and transition times that are reconfigurable during model execution for dynamic structural change – ideal for agent-based decision making. *DEVS Markov models are full-fledged DEVS models and can be coupled*

with other DEVS components in hierarchical compositions [8] to unravel the complexities of interconnected agents needed to analyze and evaluate space-enabled kill-webs.

MBSE and Science-Based Based Verification, Validation, And Accreditation (VV&A) Process

Our methods are currently being applied toward advancing verification & validation (V&V) [11-13] and test & evaluation (T&E) [14,15]. For example, our methods have been used to prove sufficient conditions for selection of verification models [12,13], which are inherently valid due to their mathematical underpinning. Our recent work, associated with the Department of Operational Test and Evaluation (DOT&E) has been leveraged to propose phases of digital transformation of T&E [3]. Our past work will be leveraged for this project, with emphasis on phases three of dT&E, where data and models are underpinned by mathematical theory.

Once applied, our frameworks will contribute toward, as examples, standardized data and models, wargaming, advanced threat modeling and analysis, loss of capability assessment and mitigation, documentation of performance, and identification patterns.

The concept of homomorphism is key to understanding the proposed approach to VV&A. Figure A1 depicts two spaces of system designs. The first one is the space of system requirements. The second one, the space of buildable system designs, reflects constraints on the available technology to build the system from available components. To establish a design that meets the requirements we construct a homomorphism from the second to the first. Given that the functional system design is a homomorphic image of the buildable system design, the functional system is said to be an implementable system design and is then used as a basis for verification. Here we adopt the definition of ***morphism*** from systems theory as defined in [8], which defines a morphism as a claim relating the equivalence of a pair of artifacts. The morphism mathematically characterizes the preservation of equivalence between a system design and a verification model. Note that the verification model (functional system design) is a mathematical abstraction of the buildable system that can subsequently be employed to perform tests and checks on proposed alternative to-be-built systems. Morphisms also propagate information gathered from results of test exercises on one model to others morphically related to it, building confidence in the family as a whole.

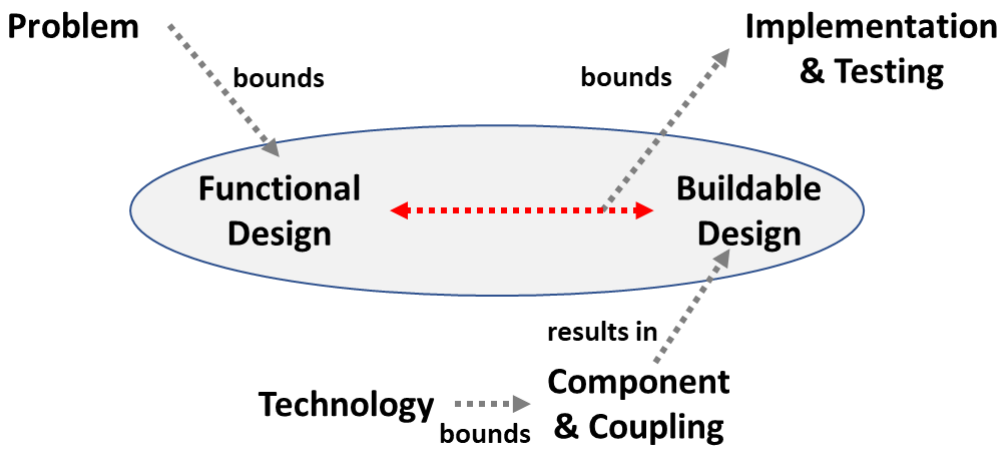


Figure A1: Illustrates the theoretical framework and concepts on which we will shape PWSA and SDA. For example, a morphism (red arrow) mathematically characterizes the preservation of equivalence between more abstract functional system design and more elaborate.

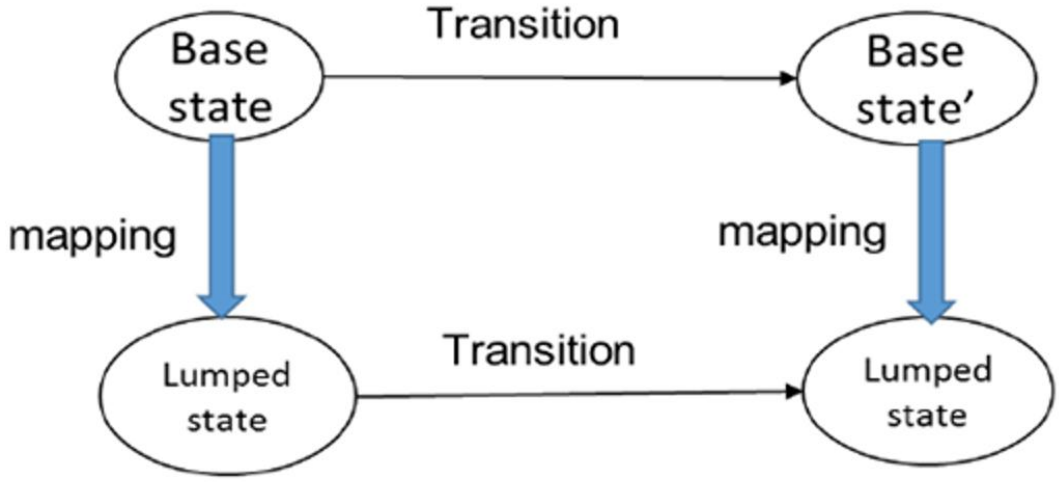
Appendix B: Background on Homomorphisms and Parameter Morphisms

A ***homomorphism*** illustrated in Figure B 1 is centered on a correspondence between base and lumped model states. The correspondence is defined by a mapping from the base model states to the lumped model states. The mapping becomes a homomorphism if it is preserved under transitions, i.e., for every base state and its corresponding lumped state the respective states to which they transit also correspond under the mapping.

In other words, identifying when a homomorphism exists means recognizing whether a structure-preserving mapping can be established between two models so that functional behaviors and system relationships are retained with respect to the questions of interest. In practice, this can carry several domain-specific meanings. Functional preservation implies that satellite tasking flows, inter-satellite links, and control paths appear equivalently in morphically related models. Design reuse occurs when a common system pattern—such as a resilient communications mesh or a tasking topology—can be applied across multiple constellation configurations. Scaling validation allows a smaller, validated model to be faithfully mapped into a larger or future PWSA model, supporting design upgrades and phase transitions. This matters because homomorphic mappings enable architecture validation, confirming that proposed updates or variations continue to support the original warfighting mission and architectural intent. They also enhance traceability and compliance by showing that new system variants remain consistent with baseline DEVS artifacts, and they provide interoperability assurance by demonstrating that coalition partner models and PWSA designs align to validate joint mission readiness.

Conversely, a homomorphism cannot be established between two system models when a structure-preserving mapping that maintains the relationships and functions of interest cannot be found. In such cases, some behaviors or connections represented in the source model cannot be faithfully carried over to the

Figure B1 Illustrating Homomorphism Concept



target model. Domain-specific meanings include structural gaps, such as missing communications links or control paths that break mission-critical connectivity; behavioral mismatches, where a satellite in one model performs functions like data relay or threat detection that have no equivalent counterpart in the other; and

broken orbital logic, where ground-to-space tasking chains or orbital adjacency assumptions fail to map, undermining coverage or coordination. This matters because, applied to mission engineering, a lack of homomorphism may indicate verification failure, indicating a new architecture version does meet the mission objectives captured in a reference model. It also can be used to detect design divergence, where differences between design intent and implementation reflect potential architecture drift or unmet requirements. Finally, it can lead to loss of simulation prediction validity, as transformations from high resolution based system models to fast running DEVS Markov models may lose accuracy, producing incorrect behavior within the intended experimental frame.

Simulation and analytic models are generally structured with parameters whose values determine their specific behaviors. ***Parameter morphisms*** are homorphisms that map parameter values from one model to the other such that the models structured with the corresponding parameter values are homorphically related.

Appendix C: Ontology and Application to PWSA

The ontology development for this project builds on past projects. A separate project was tasked with creating an initial ontology that maps systems engineering concepts to DEVS concepts. The initial ontology is represented in Table C1. Furthermore, the left column also represents the scenario from which the use case for this project was based.

***Table C1**: Mapping of PWSA loss scenario to ontology stack*

PWSA Loss Scenario	Mapping to Ontology Stack	
	Wymore_SDL	DEVS_formalisms
8x satellite NTN	Set of FSD, coupled into a BSD	Set of IO Systems, coupled into a Network of Systems
2x ground station	FSD, may be coupled into a BSD with the satellites	IO System, may be coupled into a Network of Systems with the satellites
1x unfriendly disruptor	FSD, may be coupled into a BSD with the satellites	IO System, may be coupled into a Network of Systems with the satellites
Link16 comms	Parameterize IO	Parameterized IO
Extension laser mediated communication	Parameterize IO	Parameterized IO
Nominal communication	BSD of coupled satellites and ground stations	Network of Systems of coupled satellites and ground stations
Off-nominal communication	BSD of coupled satellites, ground stations, and unfriendly disruptor	Network of Systems of coupled satellites, ground stations, and unfriendly disruptor
Rerouting	Homomorphism between Nominal communication and off-nominal communication	Homomorphism between Nominal communication and off-nominal communication

The ontology for this project focused strictly on DEVS. The basis for the DEVS ontology is [39]. To account for project resource constructs the ontology focused on defining an appropriate level of detail for the DEVS IO System (Base) model and DEVS Markov (Lumped) model. The class structure is shown in Figure C1. The object properties (i.e., relationships) are shown in Figure C2. The individuals (instantiated from the use case) are shown in Figure C3. Finally, Figure C4 and Figure C5 bring the pieces of the ontology together to show the ontological meaning of an IO System (Figure C4) and ontological meaning of a Markov model (Figure C5). The right of Figure C4 and Figure C5, captures the specific algebraic structures and instance of the two types of DEVS models.

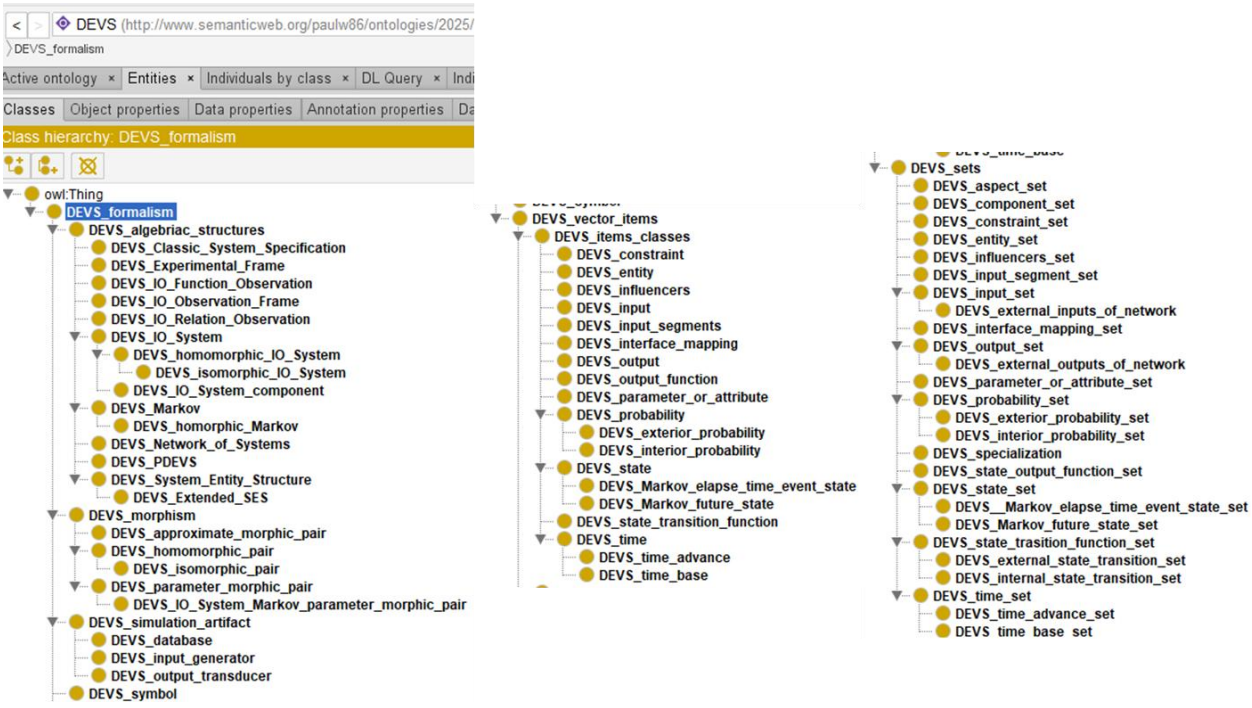


Figure C1: Class structure (i.e., taxonomy) of the DEVS ontology

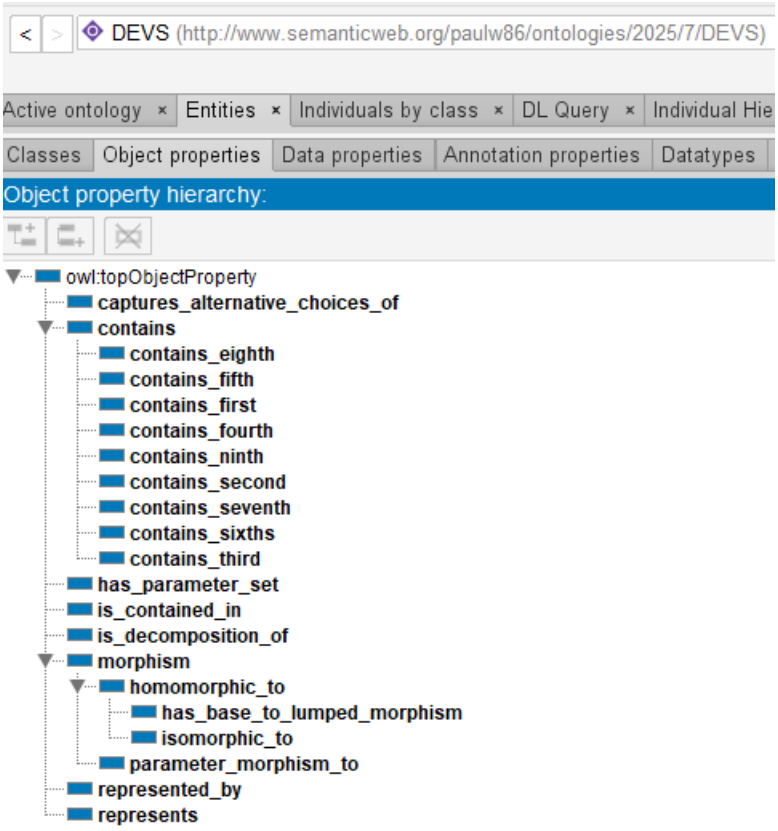


Figure C2: Object properties (i.e., possible relationships) in the DEVS ontology

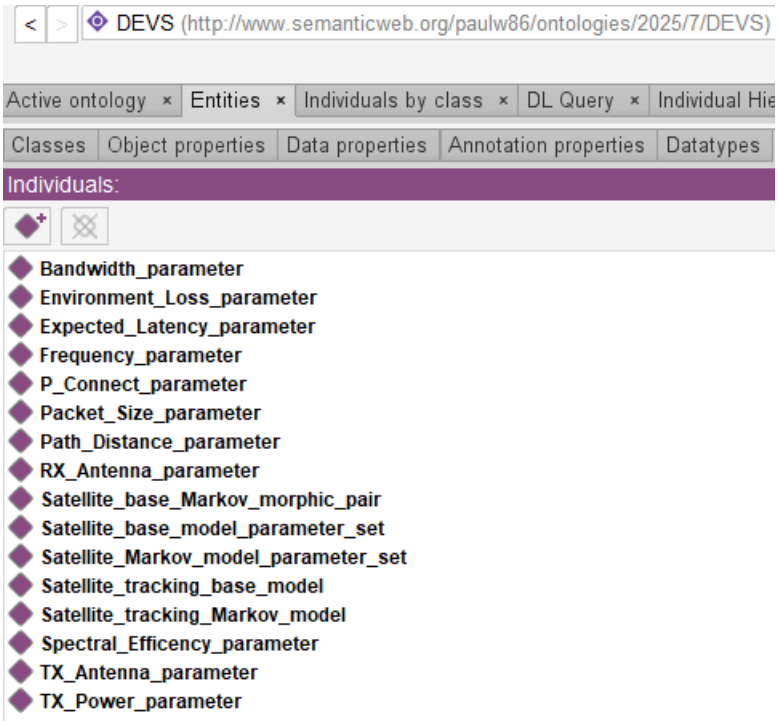


Figure C3: Instantiation of the DEVS ontology with SDA relevant context

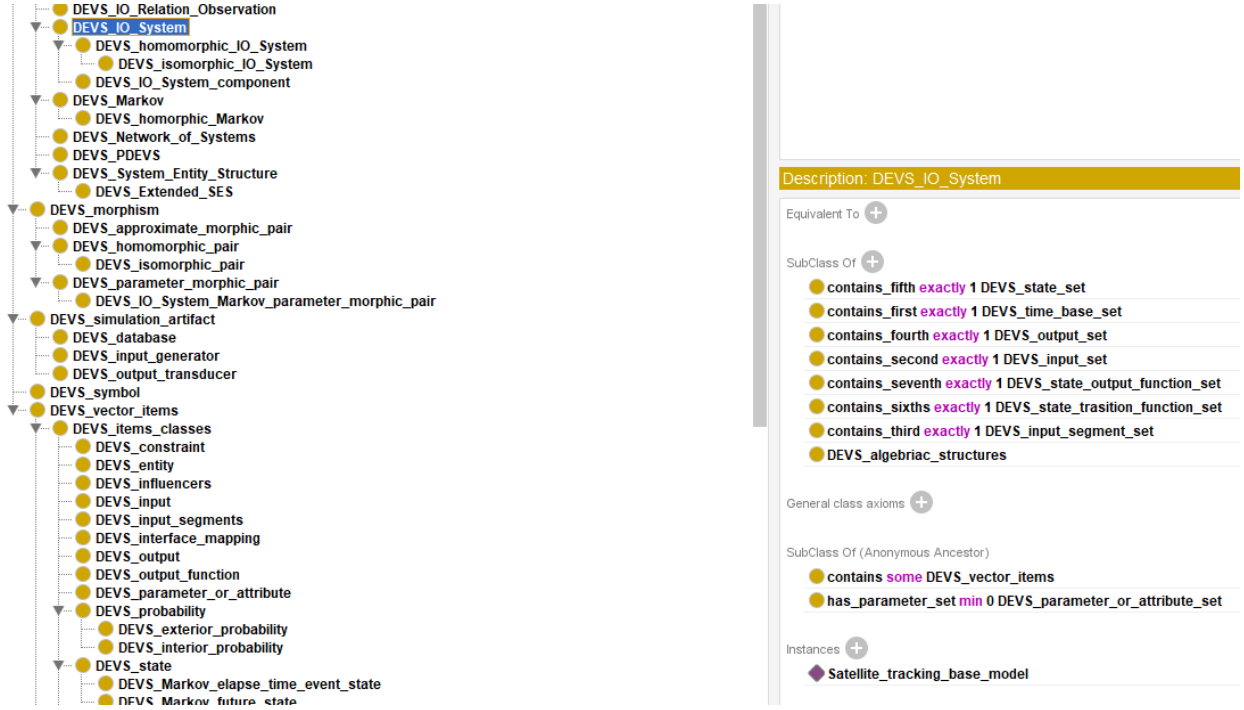


Figure C4: Detail of what constitutes a DEVS IO System model and its instance

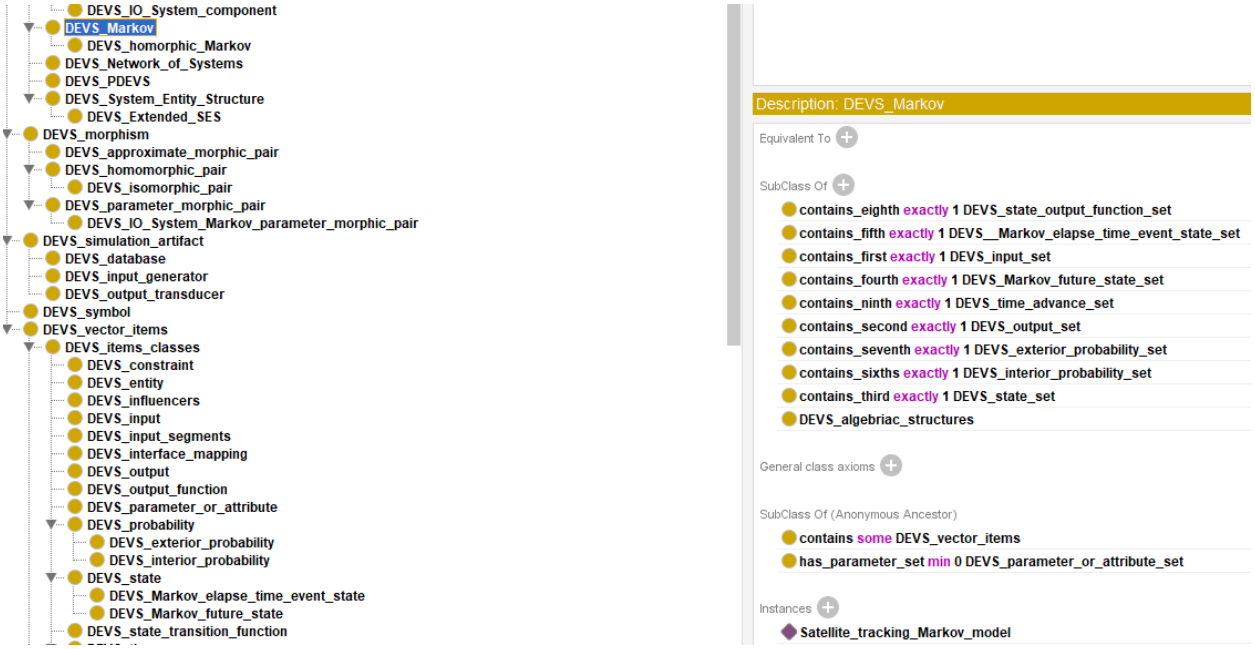


Figure C5: Detail of what constitutes a DEVS Markov model and its instance

Appendix D: Graph Databases and Algorithms

A ***graph database*** offers a means of storing structured data where nodes represent object instances and edges represent relationships between those object instances. These tools are frequently used to build knowledge graphs, social networks, fraud detection mechanisms, etc. There are a multitude of options for building a graph database such as JanusDB, GraphDB, Azure Cozmos, or Amazon Neptune, but Neo4j and ArcadeDB were chosen to be used on this project. Neo4j offers the most robust solution being a pure graph database with the most capability and market share. Those benefits come along with a license that can vary in price, so ArcadeDB was utilized both as an open-source alternative with no monetary cost as well as an evaluation mechanism to compare against Neo4j. In the case of both of these tools, they can be self-hosted which allows users to maintain full control of where their data is being stored and sent. Additionally, both of these tools have native Java support through an API package. Neo4j requires an external instance to be running, such as the Neo4j Desktop 2 application. ArcadeDB can connect to an external container instance, but it is also capable of running embedded within the Java Virtual Machine (JVM). This Java connectivity is useful for connecting with other Java native applications such as RTSync's MS4 ME modelling application and ontology development tools such as Protégé and Rosetta. Finally, many of these graph databases, including both Neo4j and ArcadeDB, utilize Cypher as the querying language for manipulating the database. The uniformity in queries simplifies the implementation and maintenance of multiple graph databases.

An ***ontology*** is a way of defining the structure and components of a complex system with many moving and interacting parts. Protégé and Rosetta are two tools used for creating and modifying ontologies. Utilizing Protégé as the main ontology editor, object classes can be defined by name, their parent class, and their components. For example, a PetMammal class might be defined as an object the has Fur and has Tail, and a Dog class could be a subclass of PetMammal. The Dog class would then inherit all properties of the PetMammal class (has Fur and has Tail) and it could have additional properties such as contains Bark. Ontologies also have the concept of an anonymous ancestor; classes that objects inherit from, but are not directly visible to the object itself. Extending the previous example, the Pet class might contain at least two Eyes and contain one Mouth. PetMammal would be a subclass of Pet. Now the Dog class will also inherit from the Pet class as an anonymous ancestor. Additionally, the Fur, Tail, Eye, and Mouth objects should also be defined elsewhere in the ontology to get a complete representation of the overall system. Object individuals can also be defined as instances of a class. For example, a Poodle might be one individual of the Dog class and a Bulldog could be another. Once these classes and individuals are defined, the ontology can be read with various tools. OWLApi is one such tools that can programmatically read ontologies with a Java package readily available. Utilizing OWLApi to read the ontology, the classes and all of their relationships can be mapped into the graph databases, and from there the individuals and all of their relationships can be mapped in as well.

Combining these tools. The DEVS ontology was written in Protégé to define each class within the DEVS framework as well as defining individual instances of the model and parameter classes. Through a Java application using OWLApi, the DEVS ontology is processed to represent each DEVS class as a node, with each subclass and component connected through an edge representing that specific relationship. Similarly, the individuals within the DEVS ontology are stored within the graph databases, so all information defined within the ontology can be accessed in a graph format.

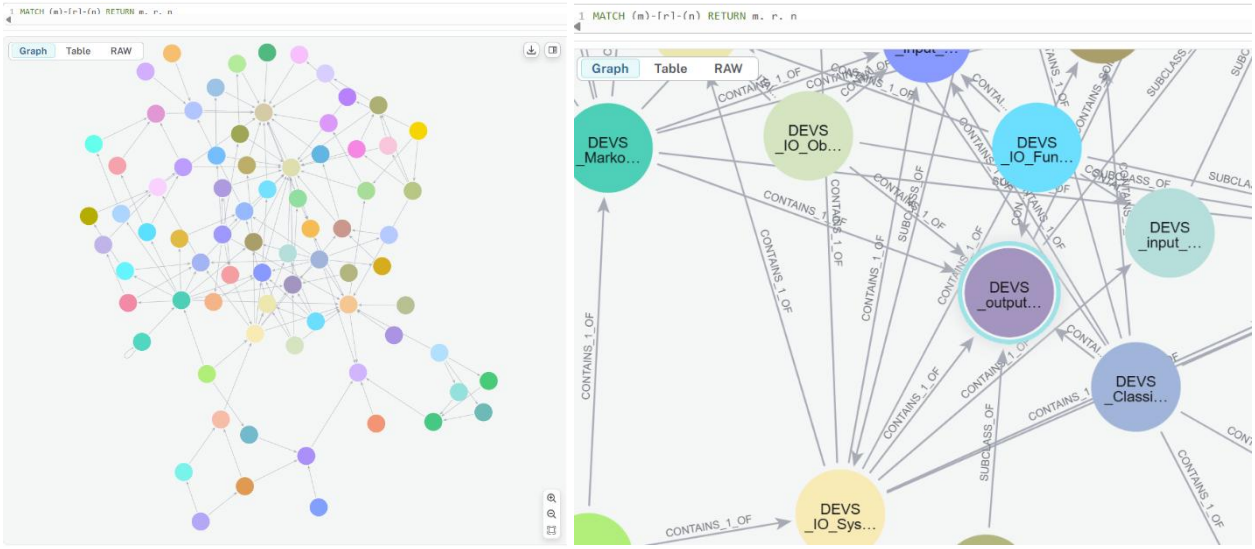


Figure D1: Above are two images of the DEVS ontology shown in a graph format. The cypher query "MATCH (m)-[r]-(n) RETURN m, r, n" is used to collect and display all nodes and relationships within the database. The left image shows a zoomed-out collection of the entire DEVS ontology, and the right image shows a zoomed in version with labelled nodes and edges.

With the DEVS ontology stored within the graph database, specific nodes and relationships can be queried. Given the name of a model individual, all of the parameters of the model can be collected along with the homomorphic relationships to other models. The name and parameters of a model can be used to create a Java Model object that is completed with a parameter map passed into the application as input. With the homomorphic relationship, a Java class can be defined with the same name to be accessed dynamically with reflection, and it would contain the programmatic means of converting one model to another. Therefore, given the ontology, input model name, input model parameters, and target model name, the application can populate the graph databases from the ontology, dynamically search for a homomorphic relationship between the input and target model, and use one or more of the found relationships to dynamically load that class that will convert the input model to the target model. Afterwards, the input and target model instances can be input into the database for later retrieval.

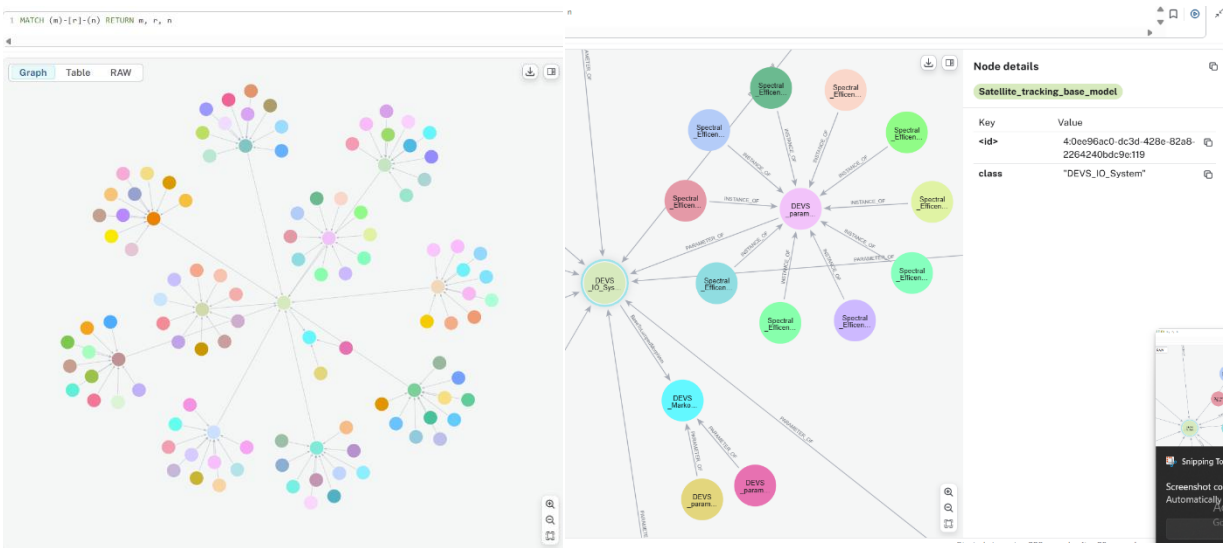


Figure D2: The graphical representation of the filled database using the same query as Figure 13. On the left is a zoomed-out image of the graph complete with the input model, input model parameters, input model parameter values, target model, and target model parameters. The right image shows a zoomed in image of the same graph with labelled nodes and edges. The edge labelled BaseToLumpedMorphism can also be seen, and this edge is extracted to identify the class name needed for Java reflection.

Extending this application further, input models might not necessarily be complete as they are initially provided. Approximation functions can also be written if one or more parameters are missing to do a similarity search on the previously inserted model instances to populate missing input fields.

Once the DEVS classes, DEVS individuals, and model instances are stored within the database, that data will persist and can be accessed when needed. This could be through the MS4 ME application to create model families, or it could be through the database user interfaces (UI) themselves to query the graphs for information.

```
[
    {
        "InstanceName": "S_Band",
        "BaseModelName": "Satellite_tracking_base_model",
        "BaseModelParameters":
        [
            {"Frequency": 2.4E9},
            {"Bandwidth": 10e6},
            {"Path_Distance": 10e3},
            {"TX_Antenna": 10},
            {"TX_Power": 30},
            {"RX_Antenna": 5},
            {"System_Noise_Temperature": 290},
            {"Environment_Loss": 3},
            {"Spectral_Efficency": 1.0},
            {"Packet_Size": 1500}
        ],
        "TargetModelName": "Satellite_tracking_Markov_model",
        "TargetModelParameters": {}
    },
```

```
[
    {
        "InstanceName": "S_Band",
        "TargetModelName": "Satellite_tracking_Markov_model
        "BaseModelParameters":
        [
            {"Packet_Size": 1500},
            {"RX_Antenna": 5},
            {"Environment_Loss": 3},
            {"TX_Antenna": 10},
            {"TX_Power": 30},
            {"Path_Distance": 10000.0},
            {"Bandwidth": 10000000.0},
            {"Frequency": 2400000000.0},
            {"System_Noise_Temperature": 290},
            {"Spectral_Efficency": 1}
        ],
        "TargetModelParameters": [
            {"P_Connect": 1},
            {"Expect_Latency": 0.0012}
        ],
        "BaseModelName": "Satellite_tracking_base_model"
    },
```


Figure D3: The input JSON (left) and the output JSON (right). Given the input JSON, the relevant information is extracted from the graph to obtain the function needed to compute the TargetModelParameters field as shown in the output.

```java
public Model exactMorphismFunction() {
    // Physical constants
    int prop_speed = 299792458;                // speed of light (m/s)
    double k_boltzmann = 1.380649e-23;         // Boltzmann constant (J/K)
    System.out.println(this.base_model.toString());
    double frequency_Hz = ((BigDecimal)this.base_model.getParameterValue("Frequency")).doubleValue();
    double bandwidth_Hz = ((BigDecimal)this.base_model.getParameterValue("Bandwidth")).doubleValue();
    int distance = ((BigDecimal)this.base_model.getParameterValue("Path_Distance")).intValue();
    int Gt_dBi = (int)this.base_model.getParameterValue("TX_Antenna");
    int Pt_dBW = (int)this.base_model.getParameterValue("TX_Power");
    int Gr_dBi = (int)this.base_model.getParameterValue("RX_Antenna");
    int Tsys_K = (int)this.base_model.getParameterValue("System_Noise_Temperature");
    int env_loss_dB = (int)this.base_model.getParameterValue("Environment_Loss");
    double spect_efficency = ((BigDecimal)this.base_model.getParameterValue("Spectral_Efficency")).doubleValue();
    int packet_bytes = (int)this.base_model.getParameterValue("Packet_Size");
    // Geometry / FSPL
    double wavelength = prop_speed / frequency_Hz;
    double FSPL_dB = 20 * Math.log10(4 * Math.PI * distance / wavelength);   // free-space path loss (dB)
    // Total losses and received power
    double L_tot_dB = FSPL_dB + env_loss_dB;  // aggregate losses (dB)
    double Pr_dBW = Pt_dBW + Gt_dBi + Gr_dBi - L_tot_dB;
    double Pr_W = Math.pow(10, (Pr_dBW / 10));
    // SNR (linear and dB) referenced to noise power in noise bandwidth B
    double NoisePower_W = k_boltzmann * Tsys_K * bandwidth_Hz;
    double SNR_linear = Pr_W / NoisePower_W;
    double SNR_dB = 10 * Math.log10(SNR_linear);
    // Eb/N0
    double bits_rate = spect_efficency * bandwidth_Hz;
    double EbN0_linear = SNR_linear / (bits_rate / bandwidth_Hz);
    double EbN0_dB = 10 * Math.log10(EbN0_linear);
    double bit_error_rate = Erf.erfc(Math.sqrt(2 * EbN0_linear) / Math.sqrt(2)) * 0.5;
    // Approximate packet error probability (PER) for a packet of N bits, PER ~ 1 - (1 - BER)^N
    double packet_bits = packet_bytes * 8;
    double packet_error_rate = 1 - Math.pow((1 - bit_error_rate), packet_bits);
    // Calculate Success Rate
    double success_rate = 1 - packet_error_rate;
    // Calculate Transmit Latency
    // Avoid division by zero
    double expected_retransmissions;
    if (packet_error_rate >= 1) {
        expected_retransmissions = Double.MAX_VALUE;
    } else {
        expected_retransmissions = 1 / (1 - packet_error_rate);
    }
    double time_prop = distance / prop_speed;
    double time_tx = packet_bits / bits_rate;
    double latency = time_prop + expected_retransmissions * time_tx;

    this.target_model.setParameter("Expect_Latency", latency);
    this.target_model.setParameter("P_Connect", success_rate);
    return this.target_model;
```

Figure D4: The calculation function of the BaseToLumpedMorphism class to determine the target model parameters

Appendix E: Agentic AI Exploration and Considerations for Future Phases

Limited research was conducted to explore future use of agentic AI. Recent years has seen the wide-scale adoption of generative AI, such as OpenAI's ChatGPT large language models (LLMs). The year 2025 has seen a transition from the passive generative AI to the active agentic AI. Agentic AI leverages LLMs as an interface to humans and as an orchestrator for an ecosystem of software platforms. Using weather as an example, generative AI will only recall weather within its knowledge base and may hallucinate (make-up) future weather predictions that could be entirely false, whereas, agentic AI can leverage the tools in its orchestration toolbox to compute future weather patterns. Therefore, agentic AI is the desired technological path for pairing with the technology being developed as part of this project.

The limited research conducted as part of this project centered the integration of LangGraph and a Model Context Protocol (MCP) to allow connections of all relevant tools in construction of the satellite system. LangGraph (i.e., the orchestration framework) may serve in managing the logic of the multi-agent workflow, while the use of MCP aides in managing the data and context that those agents share and consume. Table E1 is a current list of some of the tools envisioned to be used in the future.

Table E1. List of some potential tools expected for future use

Domain	Example Tools	Nature	Integration Needs
Signal / Comms	GNUradio	Stream-oriented	Real-time dataflow, RPC bridge
Simulation / Math	MATLAB, MS4Me	Batch/iterative	Script invocation, file or API exchange
Design / Geometry	CAD	Visual, parametric	API or model-file exchange
Semantic / Ontological	Protege (OWL)	Knowledge reasoning	RDF/SPARQL, semantic API
Graph / Database	Neo4J, ArcadeDB	Persistent, structured	REST or Bolt queries

Stemming from the list of tools, specifics of MCP can be derived such that the tools are able to effectively and efficiently interact with the LLM. Table E2 provides and overview of the MCP variations.

Table E2. MCP variations and fitness for purpose

MCP Type	Core Idea	When to Use	Fit for You

Local (monolithic)	All tools accessible in one environment via local APIs or subprocesses.	Small, single-machine setups.	Not enough — too limited for multi-domain workflows.
Service-oriented (SOA)	Each tool exposed as a network service (REST/gRPC).	Heterogeneous but static environments.	Good baseline; extend with schema registry.
Schema-driven (Semantic MCP)	Adds ontology-based schemas for describing capabilities.	When you need cross-domain reasoning.	Excellent fit — especially with Protege/Neo4J.
Event-driven / Async MCP	Uses message queues or event bus to coordinate long-running or streaming tasks.	Real-time or simulation-heavy workflows.	Needed for GNUradio and MATLAB integration.
Federated / Distributed MCP	Multiple MCP nodes each managing their own domain.	Large, multi-platform ecosystems.	Best overall structure for your use case.

The best fit was determined to be a combination of the latter three MCPs, deemed to be a "Federated Semantic MCP" that maximizes capabilities with platforms and tools for integrations and data evaluation. In order to properly integrate this MCP, a combination of multiple endpoints and MCP adapters would need to be utilized, as shown in Table E3.

Table E3. Tool and MCP pairing recommendations

Tool	MCP Adapter Type	Example Interface
GNUradio	Streaming adapter	gRPC or ZMQ endpoint; MCP node subscribes to data topics
MATLAB	Batch adapter	Python-MATLAB Engine API wrapper
CAD	File or API adapter	Calls parametric CAD API or exports via neutral file (STEP, IGES)
Protege	Semantic adapter	OWL API or SPARQL endpoint integration

Neo4J / ArcadeDB	Data adapter	Bolt or REST API query endpoint
MS4Me	Multiscale simulation adapter	SOAP/REST or co-simulation bridge via FMI/FMU standard

The specific formation of MCP enables each tool to live in its native runtime, managed independently of others. The use of the semantic type of MCP allows Protege and Neo4j to unify their data meaning, as well as bridging the physics, design, ontology, and computation of tools together under an agentic AI framework. Additional modifications could be made to provide additional capabilities, such as a Functional Mock-up Interface (FMI) to unify Matlab and MS4ME simulations as well as an RDF-based context serialization that makes every MCP exchange machine semantic.

Appendix F: Presentation Slides of the Monthly Status Meeting

SDA STTR Phase I

SF24C-T003 – SDA: Strengthening Space: Uniting Research and Innovation for PWSA (Proliferated Warfighter Space Architecture).

Contract number: FA254125PB053

DEVS Framework for Tranche-Based Mission Engineering and Performance Modeling of PWSA

Final feasibility demonstration

RTSync Corp and **VT National Security Institute**

www.rtsync.com

October 30, 2025

Distribution B: Distribution authorized to U.S. Government Agencies Only; Critical Technology. Other requests for this document shall be referred to the Contracting Officer.

Agenda

1. STTR, Government, Stakeholder Personnel
2. Scope and Objectives of Phase I
3. Task Plan Outline
4. Work completed this period
5. Work planned for next status meeting
6. Schedule of major events
7. Phase I products

Milestones and Schedule

Milestone of Tasks and Products	Deliverables	Date of Completion
Phase I Kickoff Meeting		Within 10 days of Authorization To Proceed (ATP)
Task 1. Formulate DEVS Framework for PWSA Performance Modeling	**Monthly Status Report**	**ATP + 30 days**
Task 2. Algorithm Development	**Bi-Monthly Status Report**	**ATP + 60 days**
Task 3. Develop Test Environment Task 4. Demonstrate Feasibility	**Phase I Final Report (SF 298)**	**ATP + 90 days**

NATIONAL SECURITY INSTITUTE VIRGINIA TECH

RTSync

STTR, Government, Stakeholder Personnel

Dr. Russell Teehan Mr. Richard Hencke	SDA Topic Tech Monitor	russell.f.teehan.civ@mail.mil richard.b.hencke.civ@mail.mil	To-be-updated
Dr. Bernard Zeigler	RTSync, Chief Scientist, **PI**	zeigler@rtsync.com	520-220-8811
Dr. Doohwan "DH" Kim	RTSync, Senior System Engineer, President / Project Manager	dhkim@rtsync.com	602-334-6649
Mr. Christian Koertje	RTSync, Research Staff Member	christian.koertje@rtsync.com	781-635-9513
Ms. Priscilla Carbo	RTSync, System Engineer	Priscilla.Carbo@rtsync.com	607-777-5935
Dr. Paul Wach	VT NSI, Research Assistant Professor, **Co-PI**	paulw86@vt.edu	540-232-2127
Dr. Kevin Schroeder	VT NSI, Assistant Director, Mission Systems Division	kschro1@vt.edu	540-231-7626
Alex Kyler	VT NSI	makyer19@vt.edu	
Dan DeCollo	VT NSI	ddecollo@vt.edu	
Brandt Sandman	VT NSI	bsandman@vt.edu	
Alexa Houck	VT NSI	houckao@vt.edu	
Brett Lovell	VT NSI	blovell@vt.edu	

NATIONAL SECURITY INSTITUTE VIRGINIA TECH

RTSync

DEVS-PWSA Introduction

Project Outcome

DEVS-PWSA Toolset supports Modeling, Simulation, and Analysis (MSA) for Space Development Agency emphasis on *the evaluation of space-based mission engineering and analysis.*

We have demonstrated advanced DEVS-PWSA capabilities to analyze the complex dynamics between space-based systems and their terrestrial counterparts.

PHASE I Goals and Approach

- **Use the Discrete Event System Specification (DEVS) Formalism to:**
  - Build **Base Models** for detailed component-level behavior
  - Build **Lumped Models** for aggregated system dynamics
  - Apply **Morphisms** to map between models, enabling scalable abstraction while preserving behavioral fidelity
- **Semantic Integration**
  - Ontologies: Define domain concepts and valid model transformations
  - Graph Databases: Store and retrieve model relationships efficiently
- **Mission Impact**
  - Simulate and evaluate space mission workflows
  - Analyze orbital-ground interactions
  - Support predictive decision-making for SDA initiatives

Benefits

- **Integrated Digital Engineering Platform:**
  - Full Lifecycle of Model Based System Engineering
  - DEVS-based Modeling, Simulation and Analysis
  - Ontology-driven Mission Engineering
- **Multi-Resolution Modeling Support for Digital Engineering**
- **Reusable, Composable, Scalable, and Interoperable MSA**

NATIONAL SECURITY INSTITUTE VIRGINIA TECH

RTSync

5

Scope, Technical Objectives, and Value Proposition

Scope:

Staying true to the Space Development Agency emphasis on Space Technology, *our proposed work plan aims to develop sophisticated modeling, simulation, and analysis utilities*. These tools are envisioned to *elucidate the intricate dynamics* between space mechanisms and their terrestrial counterparts *for the evaluation of space-based mission engineering and analysis.*

Primary Objectives of PHASE I:

- Elaborate a concept for a prototype, ***DEVS-PWSA***, emphasizing the application of modeling and simulation (M&S) to analyze and evaluate war-fighter relevant scenarios.
- Enhance and utilize the existing RTSync and VT-NSI infrastructure and software, leveraging their M&S, algorithmic, and System of Systems (SoS) capabilities
- Demonstrate/Illustrate the technical feasibility to align with the specified objective for one use case

Value Proposition:

- **No other [US] tool suite** inherently connects science-based models to probabilistic models and design simulations
  - ***China has adopted a comparable tool as a national standard***
  - Our tooling and methods are more encompassing
  - Enable consistency and streamlined modeling
  - Enable inherent model validation
- **Tool Uniqueness** derived from data structures based on universal mathematical underpinning
  - Enables data harmonization across a digital ecosystem
  - Enables inherent model validation and system verification
  - Enables reuse and interoperability of models and data

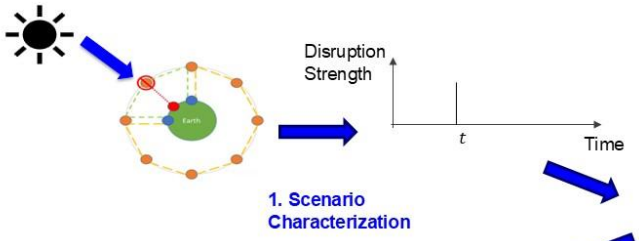
NATIONAL SECURITY INSTITUTE VIRGINIA TECH

RTSync

6

Feasibility Demonstration Goal:
Develop Scenarios for Evaluating Transport Resilience Under Solar Disruption

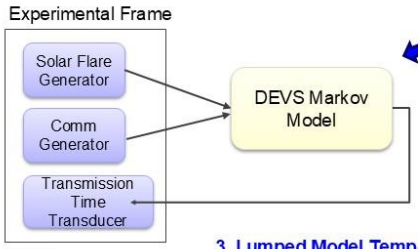
- **Scenario: Effect of solar flare disruptions over time** on satellites **along the preferred connectivity path**



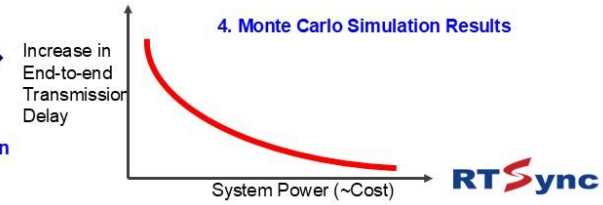
1. Scenario Characterization

	Nominal	Solar Flare (SF) (1000 SFU Event)	Mitigated w/ SF (1000 SFU Event)
Noise	Low 100 K	High 360 K	High 360 K
Power (~Cost)	Med. 10 dBW	Med. 10 dBW	High 15 dBW
Expected Latency	Low 30.12 ms	High 23,041 ms	Low 30.12 ms

2. Base Model Physics Single Hop Analysis (X_Band Comm)



3. Lumped Model Temporal Simulation



4. Monte Carlo Simulation Results

NATIONAL SECURITY INSTITUTE VIRGINIA TECH

RTSync

7

Task Plan Outline

1. **Formulate DEVS Framework for PWSA Performance Modeling (DEVS-PWSA)** (RTSync & VT-NSI Collaborative Task):
2. **Algorithms Development** (VT-NSI Responsibility): Create or prove use of existing algorithms between family of mission engineering and analysis models, which includes parameter mapping.
3. **Test Environment Development** (RTSync & VT-NSI Collaborative Task): Design and implement a conducive environment for rigorous model testing with DEVS-PWSA.
4. **Feasibility Demonstration** (RTSync & VT-NSI Collaborative Task): Validate the approach and demonstrate its applicability in real-world scenarios.

NATIONAL SECURITY INSTITUTE VIRGINIA TECH

RTSync

8

Task 1 Update: Formulate DEVS Framework for PWSA Performance Modeling

Integrate DEVS-based M&S methodology and Ontology-Driven Digital Engineering to formulate DEVS-PWSA :

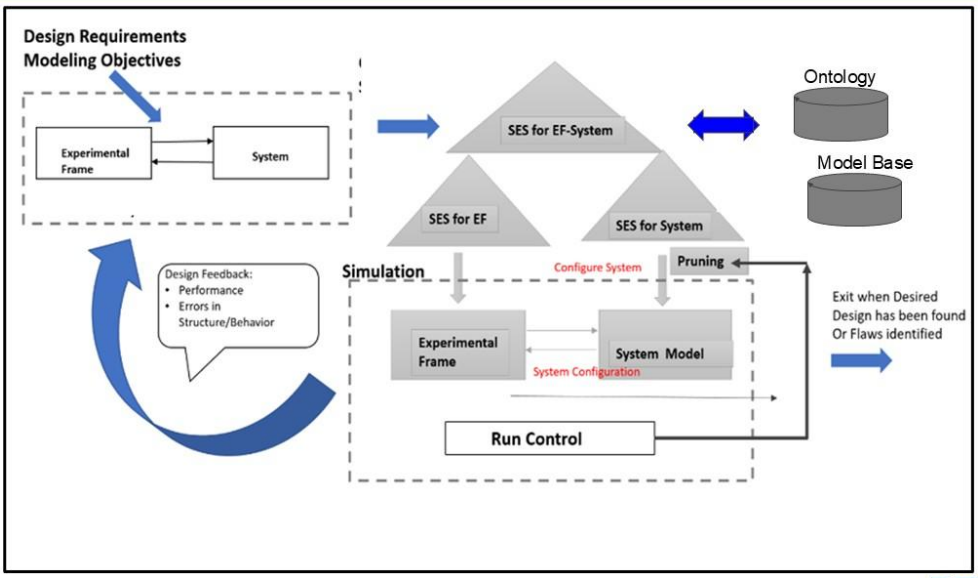
1. **Family of Models Definition**
   - Integrate DEVS morphisms and Ontology knowledge representation to manage parameters and the way they are related at different resolution levels
     - represent all capabilities of the PWSA including transport and tracking
     - range from individual satellite to constellation level metrics.
2. **Coupling Framework**
   - Apply DEVS-based approach to couple different models, supporting specific requirements to get composite models with desired performance,
3. **Scenario Definition:**
   - Apply System Entity Structure (SES) and Experimental Frames (EF) to formulate approach to modeling different environmental conditions, space vehicle anomalies, and changes in user locations.
4. **Performance Metrics:**
   - Apply SES and EF to develop consistent sets of appropriate measures of performance and effectiveness for mission threads.

NATIONAL SECURITY INSTITUTE VIRGINIA TECH

RTSync

9

DEVS Model Development Process

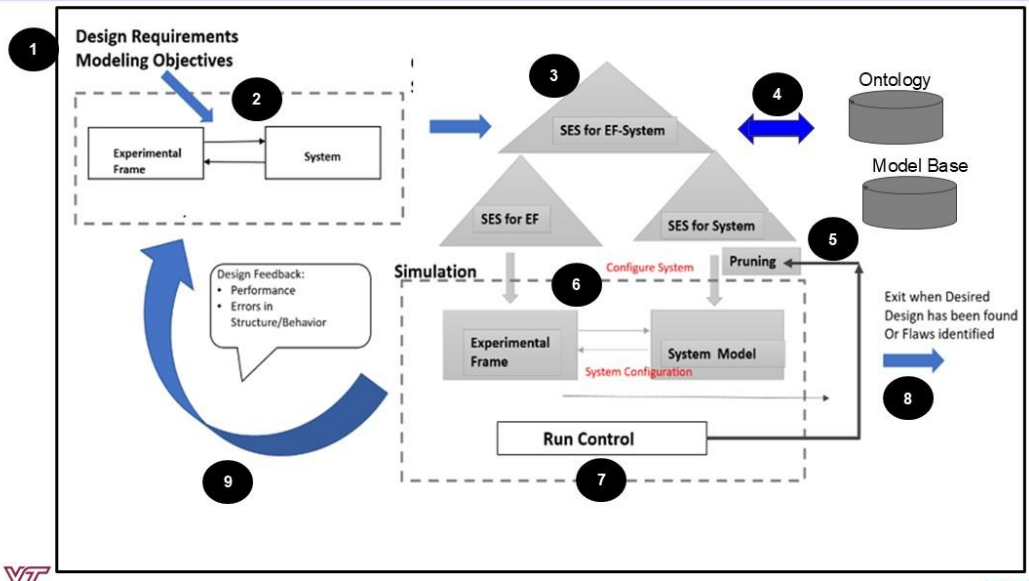


NATIONAL SECURITY INSTITUTE VIRGINIA TECH

RTSync

10

DEVS Model Development Process Flow in Order Described by Labels



NATIONAL SECURITY INSTITUTE VIRGINIA TECH

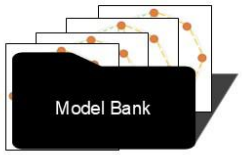
RTSync

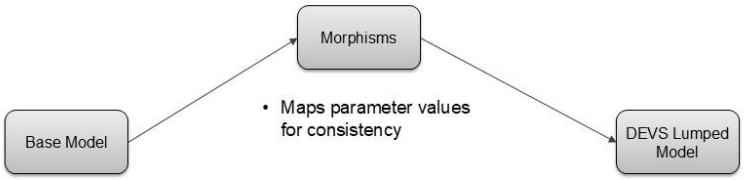
11

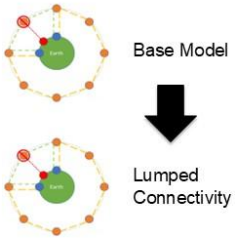
Mapping DEVS Atomic Models via Morphisms

Objective: Scenario Characterization -> Base Model Physics Single Hop Analysis -> Lumped Model Temporal Simulation

- **JSON Model Bank**
  - Store all available models with JSON key structure for easy access from ontology layer
  - Base Model, Stochastic Connectivity, Lumped Connectivity, DEVS Markov
- **JSON Morphism mappings**
  - Map Base Model parameters and their values to Corresponding Lumped Model ones







- Real system behavior
- Physical parameters

- Model of signal establishment
- Fixed, lumped parameters {PathDistance, TXPower, etc.}

NATIONAL SECURITY INSTITUTE VIRGINIA TECH

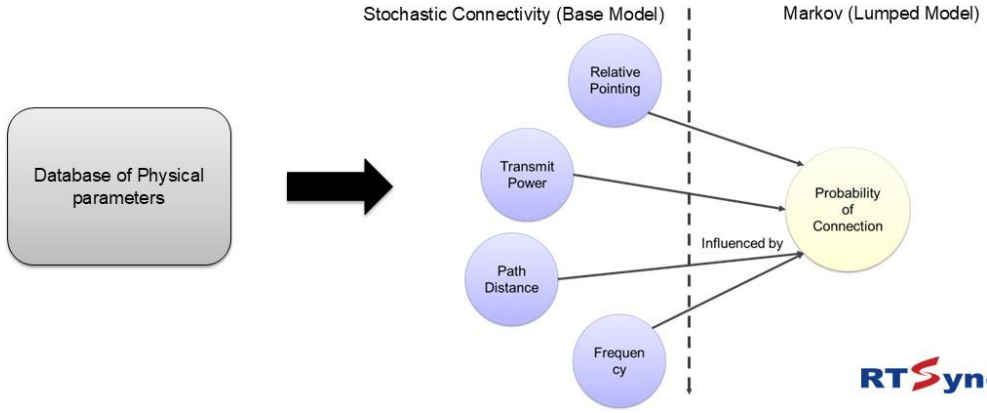
RTSync

12

Using Morphisms to Map Parameter Values

- **Example: Given a physical set up in the base model, what is the equivalent probability of connection for the Markov model?**

- **Ex: Base Model to Markov Connectivity (Shown below)**
  - Base: chosen physical parameters for the given scenario from database
  - Markov: physical parameter values are mapped to overall probability of connection

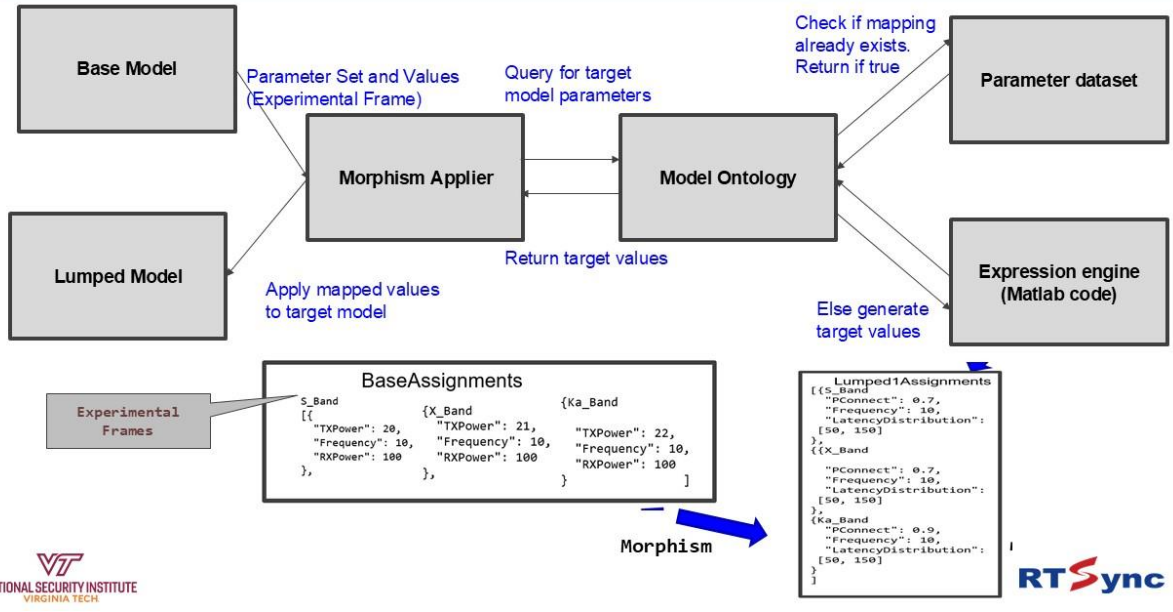


NATIONAL SECURITY INSTITUTE VIRGINIA TECH

RTSync

13

Workflow for Generation of Lumped Model Instance from Experimental Frame



NATIONAL SECURITY INSTITUTE VIRGINIA TECH

RTSync

14

Goal: Transmit Parameter Data from Base to Lumped Model

experimental frame definition

→ **scenario selection**

→ **database**

→ **morphism**

→ **lumped model values**

Inputs

- BaseModel name
- LumpedModel name
- Parameter values for chosen base

Output

- Parameter values for chosen lumped model

Parameter Morphism Example

Inputs

Output

```
{
  "CellSATToMarkovSAT":{

   "BaseName": "CellSAT",

   "LumpedName": "MarkovSAT",

   "BaseParameters": [{"PathDistance": 1e6}, {"TXPower": 10},
        {"Frequency": 1.4e10}, {"PropogationVelocity": 3e8}],

   "LumpedParameter": [{"PConnect": 0.84}, {"MeanLatency": 2.0}]
  }}
```

Forward Direction: Calibration of Lumped Model

- Parameter morphism is given base values and computes lumped values

Reverse Direction: Testing & Verification

- Lumped to Base iteration
- Parameter morphism is given lumped values and returns set of matching base values

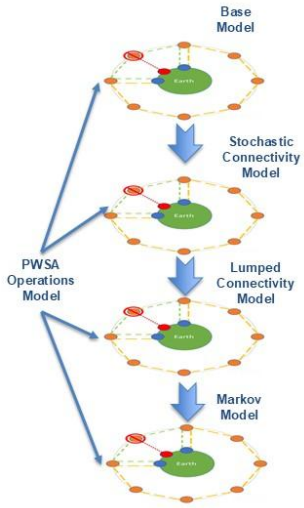
NATIONAL SECURITY INSTITUTE VIRGINIA TECH

RTSync

15

Overall Context: Family of Non-Terrestrial Network (NTN) Models

Model	Model Feature	Parameters and Relations
PWSA Operations Model	• Describes operation at the System of Systems level with component systems such as Satellite Operations, Mission Processing, Battle Management, Ground Support, and Communication Network. • Component systems provide specific services and engage in message exchanges such as mission directives and tasking, mission status updates and resource allocation.	
Base Model	Physics-based characterization of constellation structure and environment	Conceptual comprehensive system representation
Stochastic Connectivity	Predicts optimal message transmission paths under physical conditions mapped from Base Model with variance in design parameters	Random variables for connection parameters. E.g. PathDistance = N(mean, sigma)
Lumped Connectivity Model	Predicts expected message transmission paths under physical conditions mapped from Base Model	Utilizes lumped (average) values for connection variables E.g. PathDistance = <PathDistance>
Markov Model	• Stochastic model predicts SoS level statistics concerning message transmission quality (connectivity and latency)	Lumps connection variables into Probability of connection E.g. PathDistance not utilized



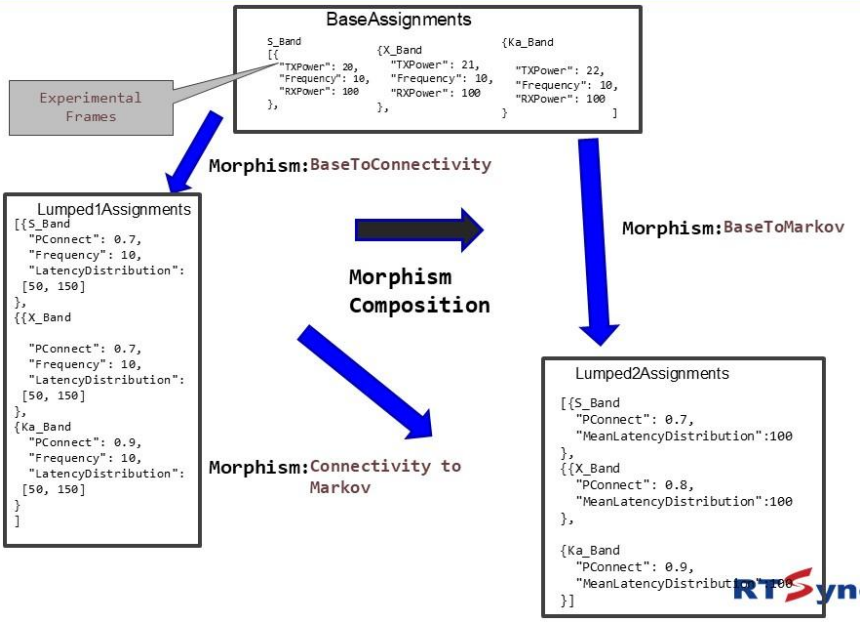
NATIONAL SECURITY INSTITUTE VIRGINIA TECH

RTSync

16

Organization of DEVS NTN Models via Morphism Composition

- **Morphisms support organization of Model Families**
- **Morphism Composition supports construction of new morphisms**



NATIONAL SECURITY INSTITUTE VIRGINIA TECH

RTSync

17

Task 2 Update: Algorithms Development (1/3)

Identifying when there is a homomorphism?

- A **homomorphism exists** when a structure-preserving mapping can be made between two models — meaning the functional behavior and system relationships are retained relative to the questions of interest
- **Potential Domain Specific Meanings**
  - **Functional preservation**: Satellite tasking flows, inter-satellite links, and control paths are all represented equivalently in morphically related models.
  - **Design reuse:** A common system pattern (e.g. resilient comms mesh or tasking topology) is reused across multiple constellation configurations.
  - **Scaling validated:** An initial smaller model can be faithfully mapped into a larger or future PWSA model — validating design upgrades or design phase transitions.

Why it matters:

- **Architecture validation:** Confirms that proposed updates or variations still support the original warfighting mission and architecture intent.
- **Traceability & compliance:** Enables verification of model lineage, showing new system variants are consistent with baseline DEVS artifacts.
- **Interoperability assurance:** Homomorphic mappings between coalition partner models and PWSA design validate joint mission readiness.

NATIONAL SECURITY INSTITUTE VIRGINIA TECH

RTSync

18

Task 2 Update: Algorithms Development (2/3)

Identifying when there is **no homomorphism** and **why it matters**

No homomorphism exists between two system models when there is no mapping that preserves structure and functionality relative to questions of interest

— i.e., some relationships or behaviors of interest in the source model cannot be matched in the target model.

Potential Domain Specific Meanings

- **Structural gaps:** A comms link or control relationship in one constellation model doesn't exist in the other — breaking mission-critical connectivity.
- **Behavior mismatch:** A satellite in the source model performs a function (e.g. data relay, threat detection) for which no equivalent component exists in the target model.
- **Orbital logic broken**: Ground-to-space tasking chains or orbital adjacency patterns don't map, violating coverage or coordination assumptions

Why it matters

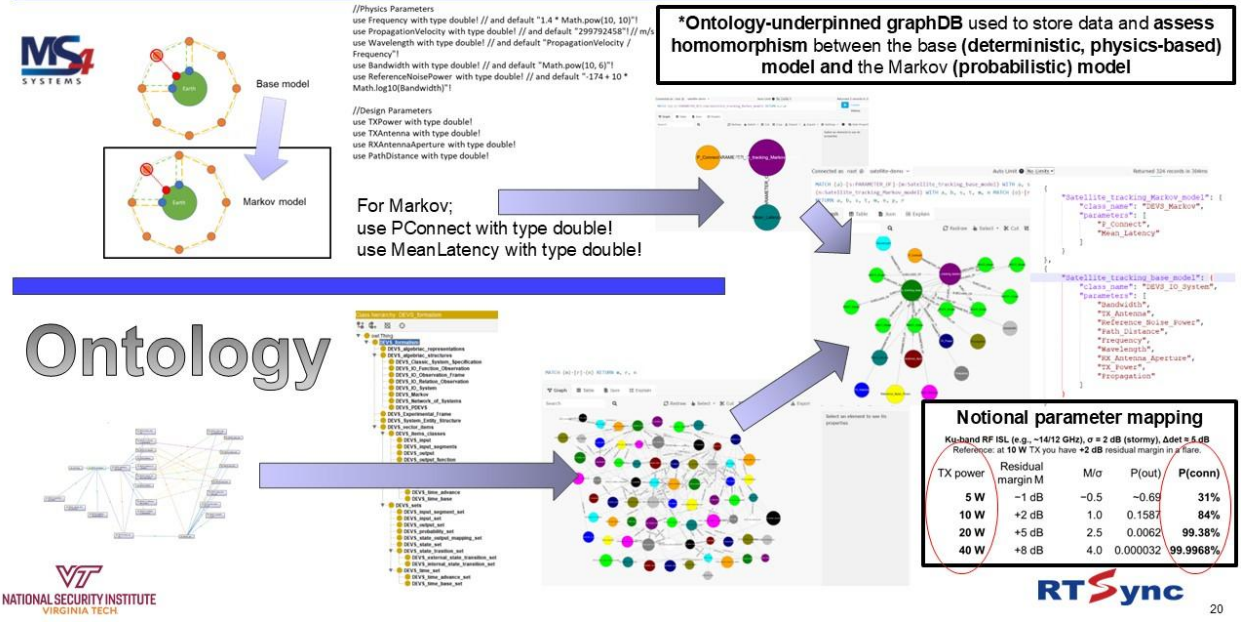
- **Verification failure:** A new architecture version cannot fulfill mission objectives of a reference model (e.g. lost resilience, degraded comms).
- **Design divergence:** Differences between design intent and implementation models may indicate architecture drift or unmet warfighter requirements.
- **Simulation breakdown:** Model transformations for DEVS (e.g. SysML → simulation graph) may lose accuracy, causing incorrect behavior in analysis tools in desired Experimental Frame

NATIONAL SECURITY INSTITUTE VIRGINIA TECH

RTSync

19

Task 2 Update: Algorithms Development (3/3)



NATIONAL SECURITY INSTITUTE VIRGINIA TECH

RTSync

20

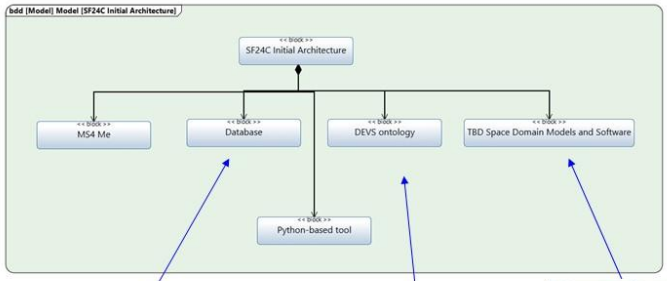
Task 3 Update: Test Environment Development (1/2)

Design and implement a conducive environment for rigorous model testing with DEVS-PWSA. Includes:

- ✓ MS4 Me
- ✓ Graph database(s)
  - ❖ Neo4j best capability, likely the most expensive
  - ❖ ArcadeDB best performance to cost
- ✓ Python-based tooling
- ✓ DEVS ontology
  - ❖ Protégé
  - ❑ Rosetta for (future goal)
- ❑ Space domain specific modeling software (future goal)

MS4 SYSTEMS

Complete

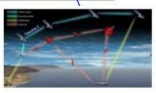


neo4j

Complete

protégé

Complete



Future goal

ArcadeDB

Complete

OpenCAESAR Rosetta

Future goal

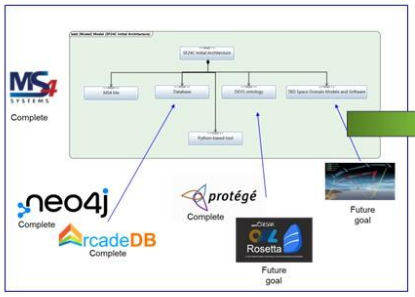
NATIONAL SECURITY INSTITUTE VIRGINIA TECH

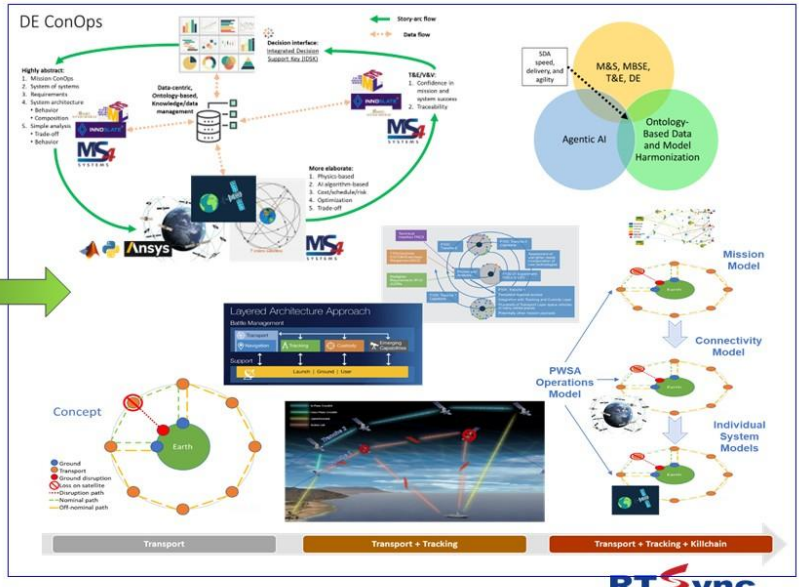
RTSync

21

Task 3 Update: Test Environment Development (2/2)

***Expected future state**
Phase 2 and beyond





NATIONAL SECURITY INSTITUTE VIRGINIA TECH

RTSync

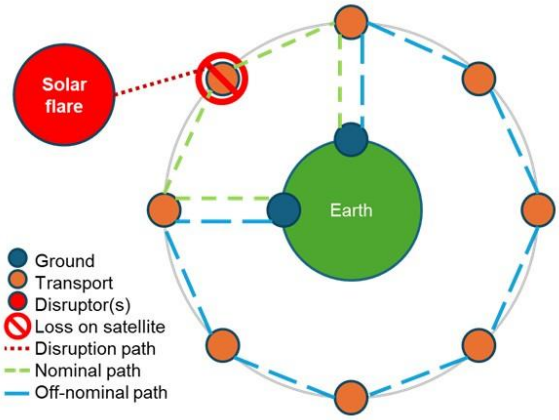
22

Task 4: Feasibility Demonstration: Scenario Formulation

Formulation for Solar flare induced disruption

Mechanism	Likely Impact on ISL
Ionization / scintillation	Amplitude/frequency fluctuations → BER spikes.
Receiver front-end noise / SEUs	Link degradation → packet errors → retransmissions.
Transponder or system outages	Complete temporary link loss → latency spikes, dropped packets.
Clock/oscillator disruption	Frequency drift → Doppler tracking errors, misalignment in phased arrays.

Propagation Delay (T_{prop})
Deterministic: $T_{prop} = \frac{d}{c}$
Depends on satellite altitude and ground distance.
Transmission Delay (T_{tx})
Time to push data onto the link: $T_{tx} = \frac{Packet\ Size}{Link\ Rate}$
Can be assumed fixed
Optionally, link rate can decrease if losses reduce SNR and you have to drop modulation order (adaptive coding/modulation).
Processing/Queuing Delay (T_{proc})
Includes coding/decoding, interleaving, protocol processing.
Often a function of **error correction strength** or **link SNR**.
Retransmission Delay (T_{ARQ})
Triggered if Bit error rate (BER, which is a function of SNR) exceeds a threshold.
Depends on the packet error probability $P_e(SNR)$, which is derived from your link budget.
As placeholder this can be modeled as expected number of retransmissions = $1/(1-P_e)$.



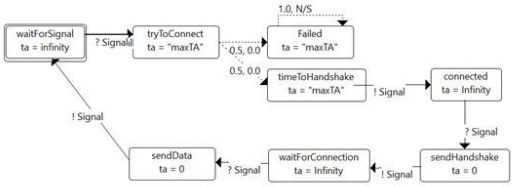
NATIONAL SECURITY INSTITUTE VIRGINIA TECH

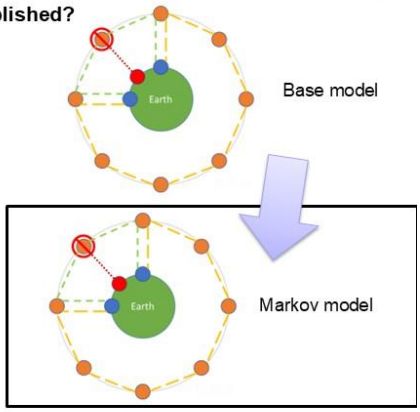
RTSync

23

Task 4: Feasibility Demonstration: DEVS Markov Approach to Satellite Connectivity

- **What is the probability of a successful network route connection being established?**





- **Dynamically set Markov transition probability and temporal distributions**
  - Pconnect can be drastically reduced following an interference
  - Pconnect may come from other model data (morphisms)

```
TransitionInfo ti;

double pConnect = 0.95; // arbitrary value for now
double pFail = 1 - pConnect;

ti = ctm.getTransitionInfoFor("tryToConnect", "Failed");
ti.setProbValue(pFail);

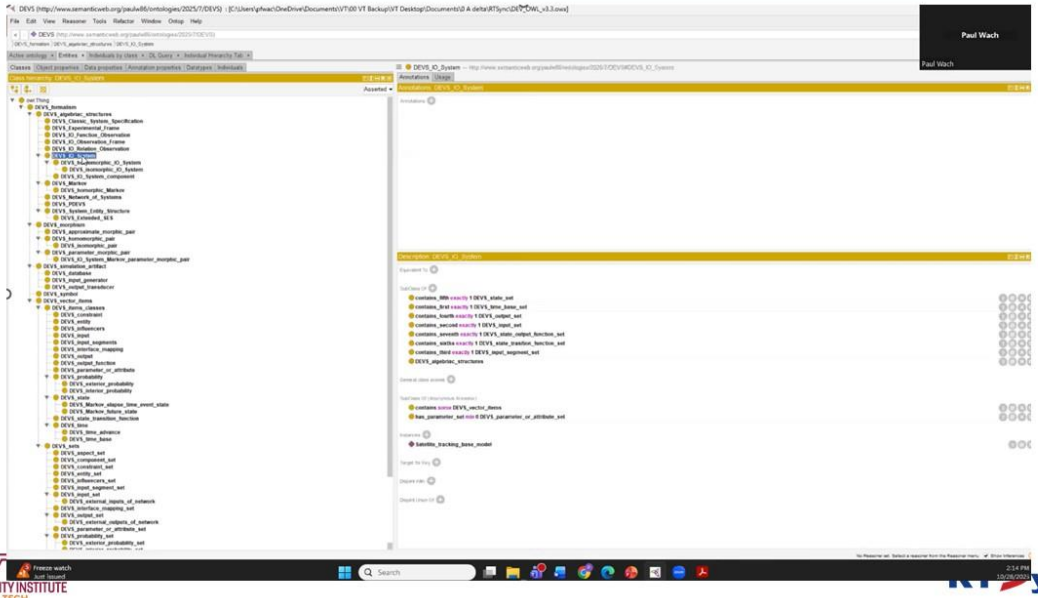
ti = ctm.getTransitionInfoFor("tryToConnect", "timeToSend");
ti.setProbValue(pConnect);
```

NATIONAL SECURITY INSTITUTE VIRGINIA TECH

RTSync

24

Task 4: Feasibility demonstration (x/y) Ontology demo

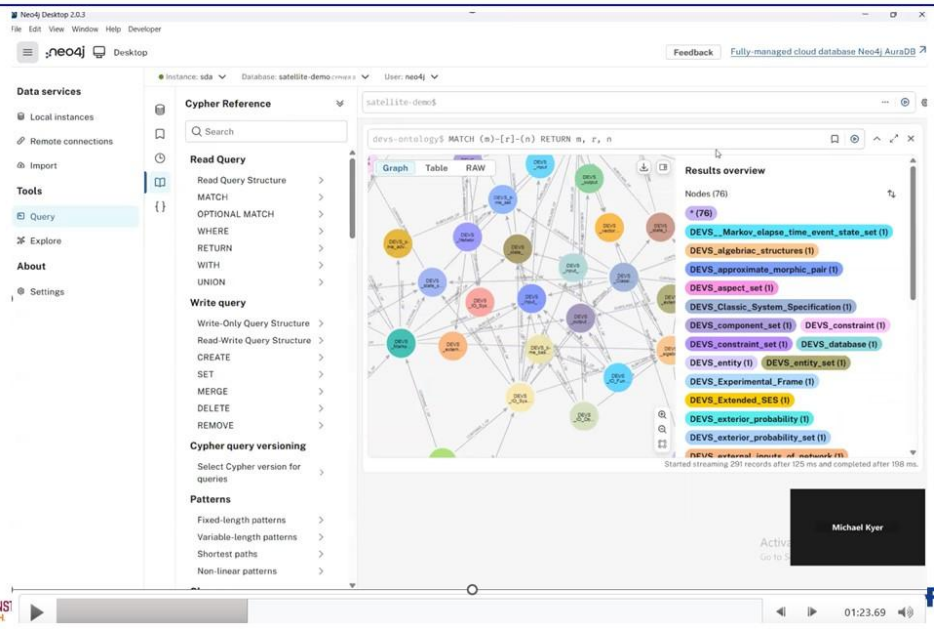


NATIONAL SECURITY INSTITUTE VIRGINIA TECH

RTSync

25

Task 4: Feasibility demonstration (x/y) Neo4j

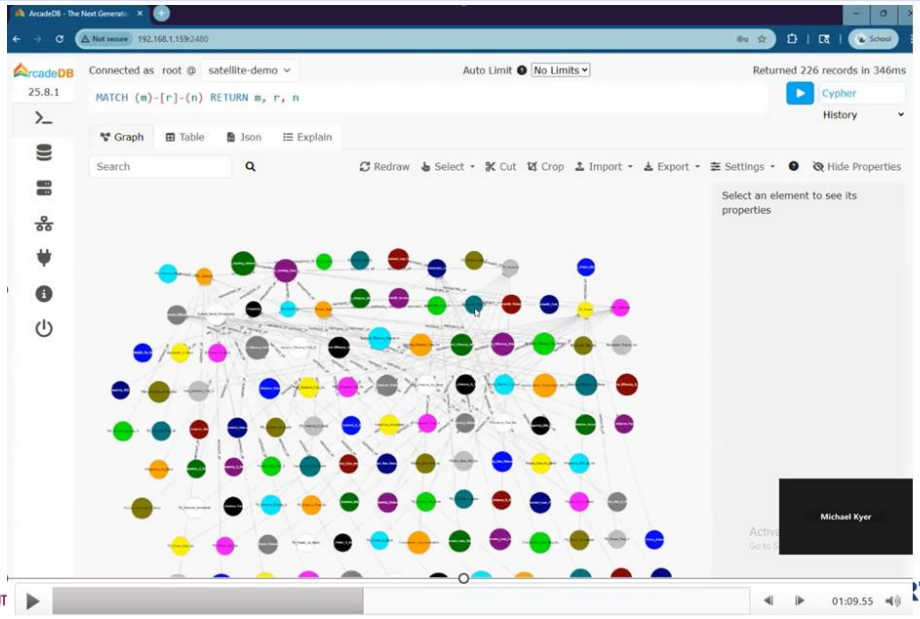


NATIONAL SECURITY INSTITUTE VIRGINIA TECH

RTSync

26

Task 4: Feasibility demonstration (5/5) ArcadeDB



NATIONAL SECURITY INSTITUT VIRGINIA TECH

RTSync

27

Task 4: Feasibility Demonstration, Results of Solar Flare Simulation

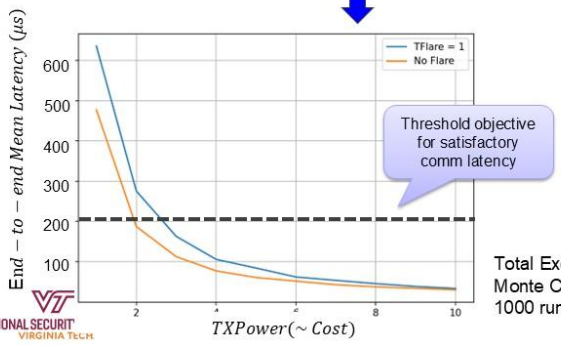
- **Comm message sent out at t = 0**
- **Solar flare at t = t_flare**

```
external event with Flare
<%
// LatencyDegredationFactor comes from morphism
System.out.println("Incident solar flare at time t = " + currentTime);
MeanLatency = LatencyDegredationFactor * MeanLatency;

TransitionInfo t
ti = ctm.getTransitionInfoFor("tryToConnect", "timeToHandshake");
TimeInfo timeInfo = ti.getTInfo();
timeInfo.setMean(MeanLatency);
ti.setTInfo(timeInfo);
%>!
```

lookup table value for increased noise environment

Flare event increases single hop mean latency



Based on X_Band example mappings

	Nominal	Solar Flare (SF) (1000 SFU Event)	Mitigated w/ SF (1000 SFU Event)
Noise	Low 100 K	High 360 K	High 360 K
Power (~Cost)	Med. 10 dBW	Med. 10 dBW	High 15 dBW
Expected Latency	Low 30.12 ms	High 23,041 ms	Low 30.12 ms

System_Noise_Temp

Base Model

Lumped Model

System normally performs well → Increased noise drastically increases time to transmit → Slight increase in power/cost can mitigate noise losses

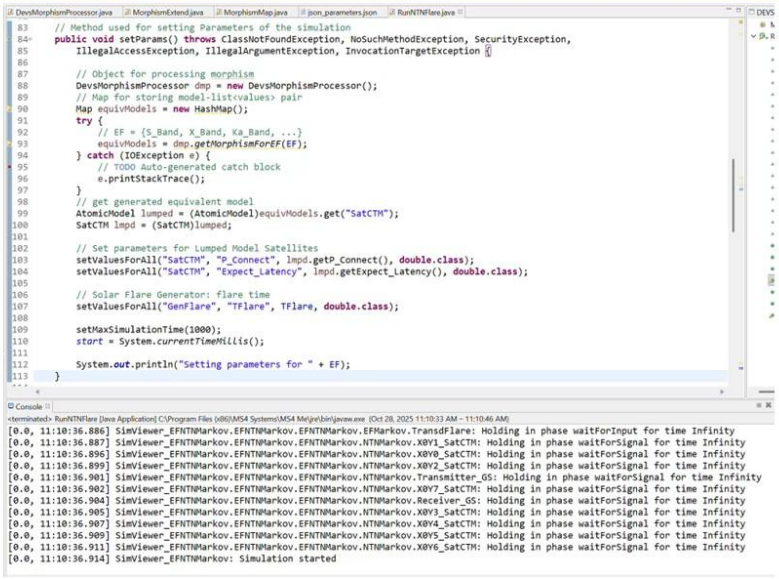
Total Execution time: 2.5 minutes
Monte Carlo:
1000 runs per TXPower, 16 secs/run

NATIONAL SECURITY INSTITUT VIRGINIA TECH

RTSync

28

Temporal Simulation of Markov Model in RTSync's Modeling and Simulation Tool



```
// Method used for setting Parameters of the simulation
public void setParams() throws ClassNotFoundException, NoSuchMethodException, SecurityException,
    IllegalAccessException, IllegalArgumentException, InvocationTargetException {

    // Object for processing morphism
    DevsMorphismProcessor dmp = new DevsMorphismProcessor();
    // Map for storing model-list<values> pair
    Map equivModels = new HashMap();
    try {
        // EF = {S_Band, X_Band, Ka_Band, ...}
        equivModels = dmp.getMorphismForEF(EF);
    } catch (IOException e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    }
    // get generated equivalent model
    AtomicModel lumped = (AtomicModel)equivModels.get("SatCTM");
    SatCTM lmpd = (SatCTM)lumped;

    // Set parameters for Lumped Model Satellites
    setValuesForAll("SatCTM", "P_Connect", lmpd.getP_Connect(), double.class);
    setValuesForAll("SatCTM", "Expect_Latency", lmpd.getExpect_Latency(), double.class);

    // Solar Flare Generator: flare time
    setValuesForAll("GenFlare", "TFlare", TFlare, double.class);

    setMaxSimulationTime(1000);
    start = System.currentTimeMillis();

    System.out.println("Setting parameters for " + EF);
}
```

```
[0.0, 11:10:36.886] SimViewer_EFNTNMarkov.EFNTNMarkov.EFNTNMarkov.EFMarkov.TransdFlare: Holding in phase waitForInput for time Infinity
[0.0, 11:10:36.887] SimViewer_EFNTNMarkov.EFNTNMarkov.EFNTNMarkov.NTNMarkov.X0Y1_SatCTM: Holding in phase waitForSignal for time Infinity
[0.0, 11:10:36.896] SimViewer_EFNTNMarkov.EFNTNMarkov.EFNTNMarkov.NTNMarkov.X0Y0_SatCTM: Holding in phase waitForSignal for time Infinity
[0.0, 11:10:36.899] SimViewer_EFNTNMarkov.EFNTNMarkov.EFNTNMarkov.NTNMarkov.X0Y2_SatCTM: Holding in phase waitForSignal for time Infinity
[0.0, 11:10:36.901] SimViewer_EFNTNMarkov.EFNTNMarkov.EFNTNMarkov.NTNMarkov.Transmitter_GS: Holding in phase waitForSignal for time Infinity
[0.0, 11:10:36.902] SimViewer_EFNTNMarkov.EFNTNMarkov.EFNTNMarkov.NTNMarkov.X0Y7_SatCTM: Holding in phase waitForSignal for time Infinity
[0.0, 11:10:36.904] SimViewer_EFNTNMarkov.EFNTNMarkov.EFNTNMarkov.NTNMarkov.Receiver_GS: Holding in phase waitForSignal for time Infinity
[0.0, 11:10:36.905] SimViewer_EFNTNMarkov.EFNTNMarkov.EFNTNMarkov.NTNMarkov.X0Y3_SatCTM: Holding in phase waitForSignal for time Infinity
[0.0, 11:10:36.907] SimViewer_EFNTNMarkov.EFNTNMarkov.EFNTNMarkov.NTNMarkov.X0Y4_SatCTM: Holding in phase waitForSignal for time Infinity
[0.0, 11:10:36.909] SimViewer_EFNTNMarkov.EFNTNMarkov.EFNTNMarkov.NTNMarkov.X0Y5_SatCTM: Holding in phase waitForSignal for time Infinity
[0.0, 11:10:36.911] SimViewer_EFNTNMarkov.EFNTNMarkov.EFNTNMarkov.NTNMarkov.X0Y6_SatCTM: Holding in phase waitForSignal for time Infinity
[0.0, 11:10:36.914] SimViewer_EFNTNMarkov: Simulation started
```

NATIONAL SECURITY INSTITUTE
VIRGINIA TECH

RTSync

29

Screenshots of Workflow

```
// Method used for setting Parameters of the simulation
public void setParams() throws ClassNotFoundException, NoSuchMethodExc
    IllegalAccessException, IllegalArgumentException, InvocationTarget

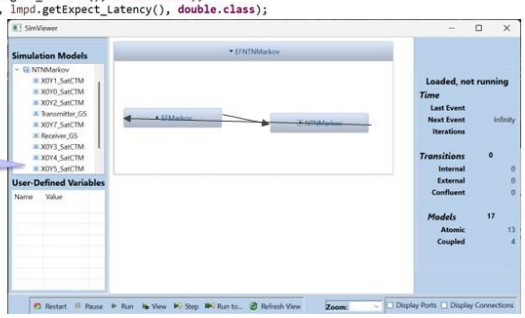
    // Object for processing morphism
    DevsMorphismProcessor dmp = new DevsMorphismProcessor();
    // Map for storing model-list<values> pair
    Map equivModels = new HashMap();
    try {
        // EF = {S_Band, X_Band, Ka_Band, ...}
        equivModels = dmp.getMorphismForEF(EF);
    } catch (IOException e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    }
    // get generated equivalent model
    AtomicModel lumped = (AtomicModel)equivModels.get("SatCTM");
    SatCTM lmpd = (SatCTM)lumped;

    // Set parameters for Lumped Model Satellites
    setValuesForAll("SatCTM", "P_Connect", lmpd.getP_Connect(), double.class);
    setValuesForAll("SatCTM", "Expect_Latency", lmpd.getExpect_Latency(), double.class);
```

Setting Parameters from Morphism JSON Processor

```
  "TargetModelParameters":   [
    {
      "P_Connect":       0.9999999671089773
    }
    {
      "Expect_Latency":       2.4000000789384569E-4
    }
  ]
  "BaseModelName":   Satellite_tracking_base_model
}
[{Packet_Size=1500.0}, {RX_Antenna=20.0}, {Environment_Loss=0.0},
Base and Lumped have consistent assignments for X_Band
Setting parameters for X_Band
Disable Logging : false
[0.0, 11:25:30.333] SimViewer_EFNTNMarkov: Simulation starting
```

Generated lumped model instance for simulation



Get results from EF

```
TStart = 0.0
TFlare = 1.0E-4
TReport = 0.0013298887932134706
Success = true
```

NATIONAL SECURITY INSTITUTE
VIRGINIA TECH

RTSync

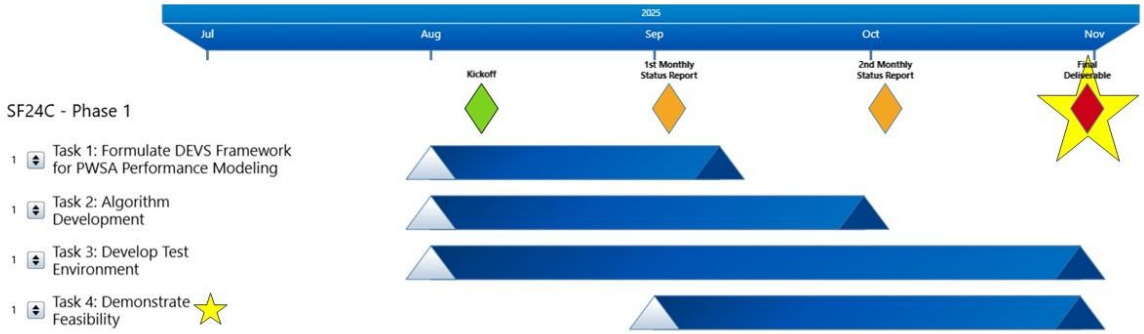
30

Summary of work completed

- **Task 1:** Completed the framework
- **Task 2:** Completed algorithm development
- **Task 3:** Completed the test environment
- **Task 4:** Completed the demonstration

October 30, 2025

1. **Final feasibility demonstration**
   - Demonstrated impacts of solar flare event to mission success based on mathematical relationships between physics-based (Base) models and probabilistic (Lumped) Markov models



NATIONAL SECURITY INSTITUTE VIRGINIA TECH

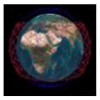
RTSync

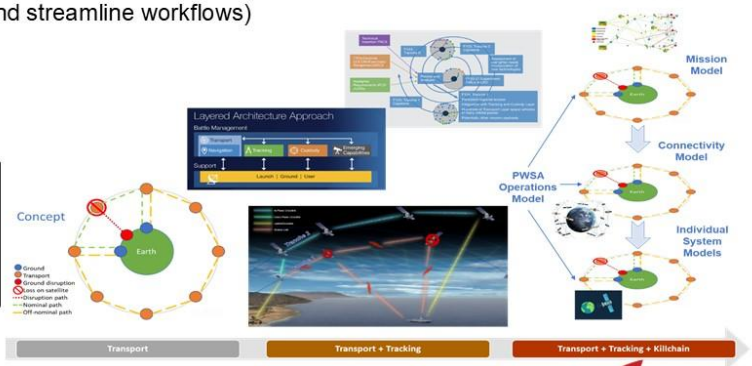
31

SOW for Phase 2 (1/2)

Enhance validation of the approach and demonstrate its applicability in real-world scenarios.

- Demonstrate **increased scenario complexity**
  - Probabilistic models and cost simulations are backed by physics (e.g., 2D to 3D network)
  - Combine transport with tracking (followed by inclusion of killchain)
  - Digital ecosystem (i.e., harmonize and streamline workflows)





***Other tools do not offer** our capability

Science-based, ontology-underpinned graphDB used to store data and **assess homomorphism** between the base **(deterministic, physics-based) model and** the Markov **(probabilistic) model**

NATIONAL SECURITY INSTITUTE VIRGINIA TECH

RTSync

32

SOW for Phase 2 (1/2)

Suggested Tasks

I. Develop use case To include
- ❑ Alternative solar flare mitigation designs with costs
- ❑ transport and tracking

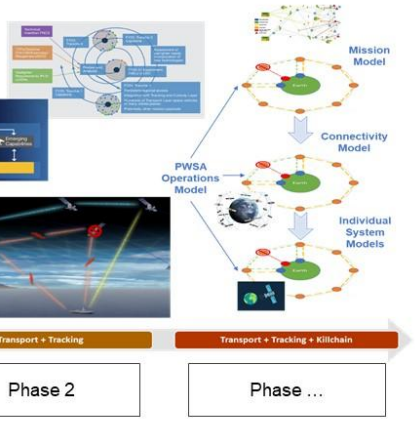
II. Develop models
- ❑ From simple, 2D models to multidimensional networks

III. Develop algorithms
- ❑ To include algebraic structure mapping

IV. Develop test environment
- ❑ To include streamlined workflows

V. Demonstrate more practical capability
- ❑ Realistic in comparison to SDA





NATIONAL SECURITY INSTITUTE VIRGINIA TECH

RTSync

33

Questions/Comments

NATIONAL SECURITY INSTITUTE VIRGINIA TECH

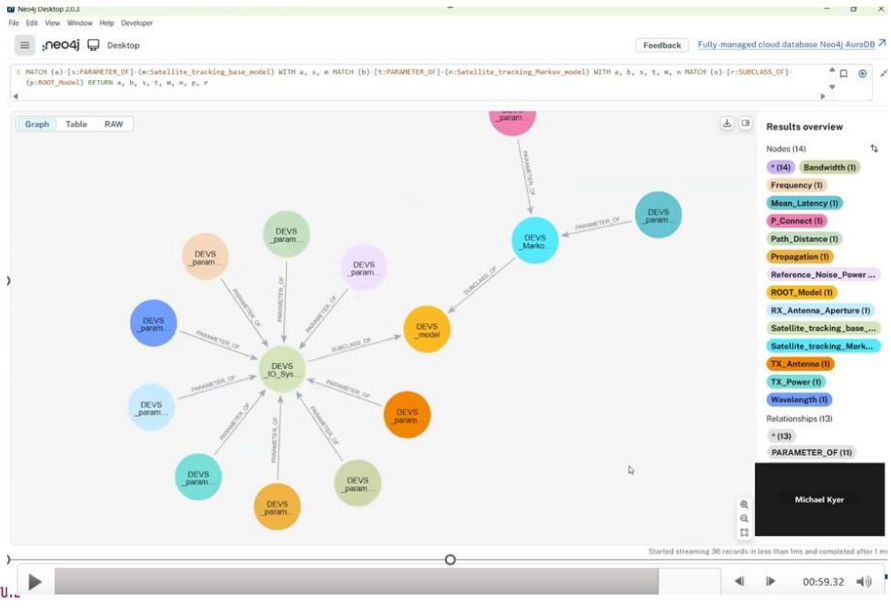
RTSync

34

BACK-UP

Includes slides from the kick-off meeting

Task 2: Algorithms Development (Previous demo)

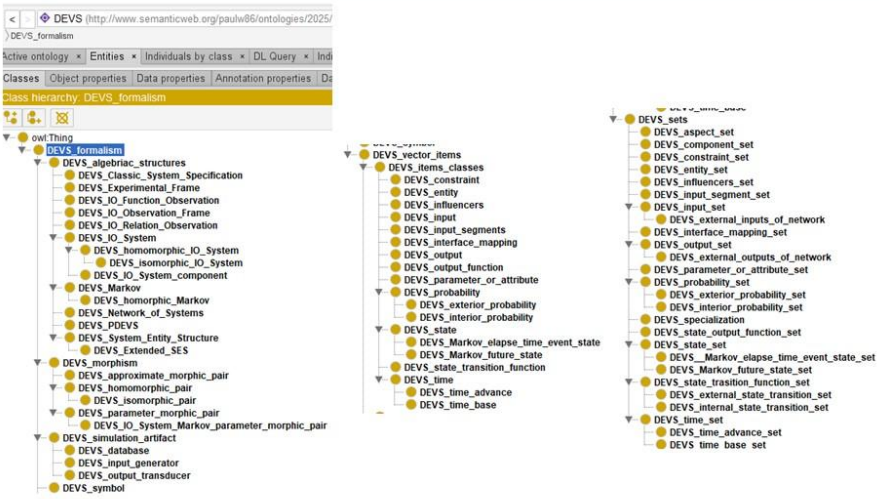


NATIONAL SECURITY INSTITU... VIRGINIA TECH

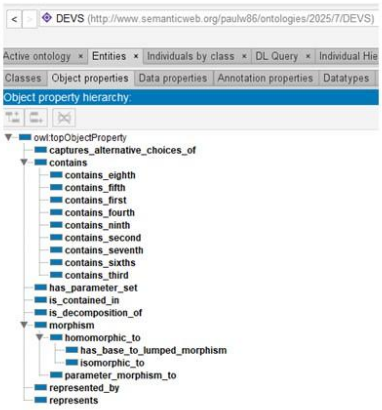
RTSync

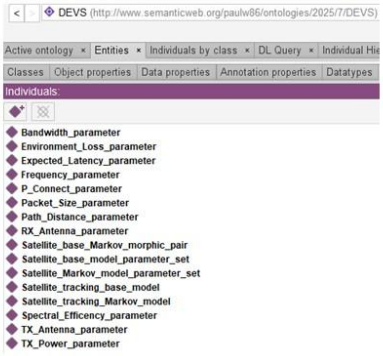
36

Ontology



Ontology





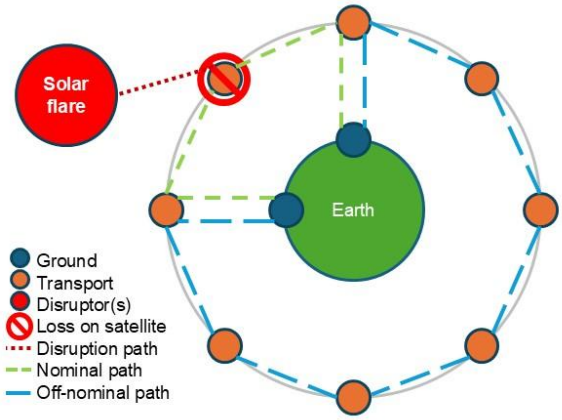
Ontology



NATIONAL SECURITY INSTITUTE
VIRGINIA TECH

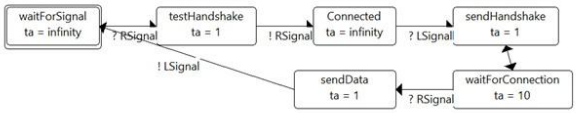
RTSync

39



DEVS Satellite Model Implements Physically Realistic Connection Logic

DEVS event handling



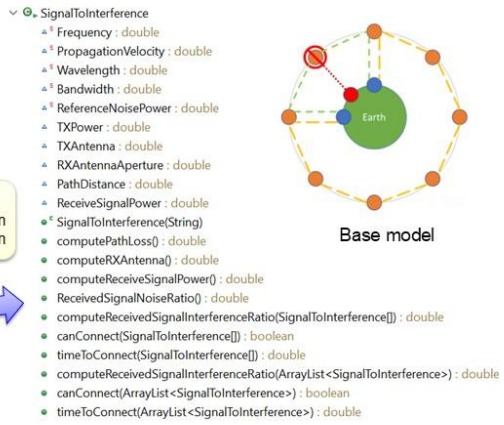
Satellite State Diagram

```
external event for waitForSignal with Signal
<%//Add your own code
//Serializable variable = messageList.get(0).getData();
//compute signal / interference
MyMessage = input;
if (SpaceVehicle.canConnect(InterfererList))
    holdIn("testHandshake", SpaceVehicle.timeToConnect(InterfererList));
else
    passivateIn("waitForSignal");
%>!
```

Call on physics-based computation to decide transition

Example of transitional event logic

Physics based computations



Base model

NATIONAL SECURITY INSTITUTE VIRGINIA TECH

RTSync

41

Examined Potential Integration of Higher Resolution Model/Test Environment

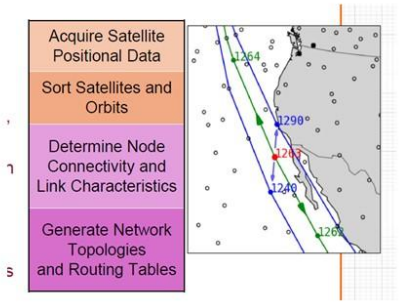
Network Testbed for Multi-Constellation Networks *

- Communications of internet satellite constellations
- Space-based high-bandwidth networking testbed
- Capability to emulate multi-constellation networks
- Resiliency study was performed for a NYC to London route utilizing the existing Starlink satellite orbits.
- Wireless Channel Loss related-Parameters

Features	Parameters	Features	Parameters
ChannelFreq_isls	37.0 GHz	Satellite_eirp	80.9 W
ChannelFreq_sat_to_ground	12.7 GHz	Satellite_eirp_dbW	50.9 dBW
ChannelFreq_ground_to_sat	14.5 GHz	Ground_station_tx_power	36.0 dBm
Channnel_bandwidth_downlink	240 MHz	Ground_station_rx_attenna_gain	33.2 dBi
Channnel_bandwidth_uplink	60 MHz	Ground_station_tx_attenna_gain	34.6 dBi
Polarization_loss	3 dBi	UE _tx_power	23 & 33 dBm
Misalignment_attenuation_losses	0.5 dB	UE_attenna_gain	0 & 39.7 dBi (rx), 0 & 43.2 dBi (tx)

Address

- high velocity satellites creating constant orbit changes
- Inter-satellite links



* Bruce Barbour et al, Network Testbed for Small Satellites (NeTSat) - Distributed Space Adaptive Communications and Security for Multi-Constellation Networks
https://doi.org/10.2514/6.2023-1502

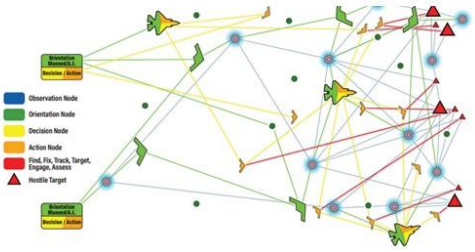
Samantha Kenyon et al, SpaceNet: Distributed Space and Terrestrial Networking Infrastructure for Multi-Constellation Coexistence

NATIONAL SECURITY INSTITUTE VIRGINIA TECH

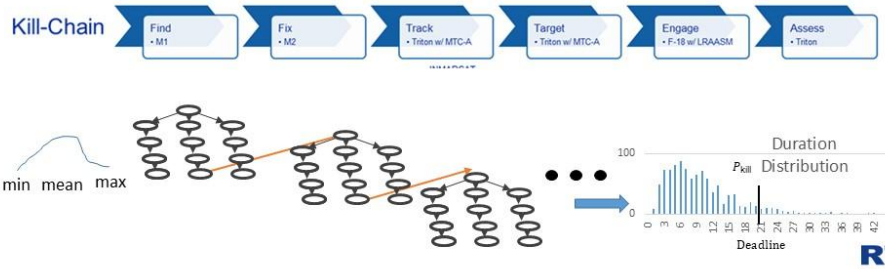
RTSync

42

Synthesis of Kill-Chains as DEVS Markov Coupled Models Example



Stochastic abstraction of kill-webs synthesized by coupling of asset descriptions extracted from a high-fidelity knowledge base using a composition of DEVS Markov models within each stage of the (dynamically) constructed kill-chain.



NATIONAL SECURITY INSTITUTE
VIRGINIA TECH

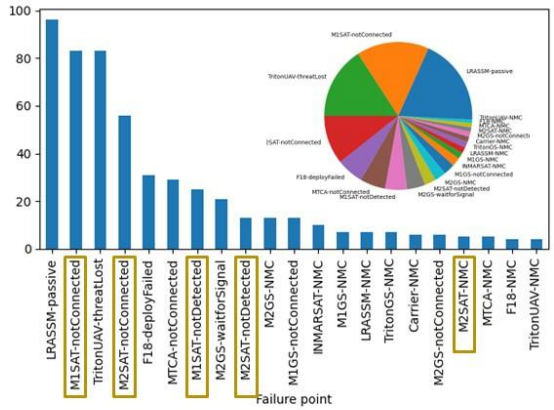
RTSync

43

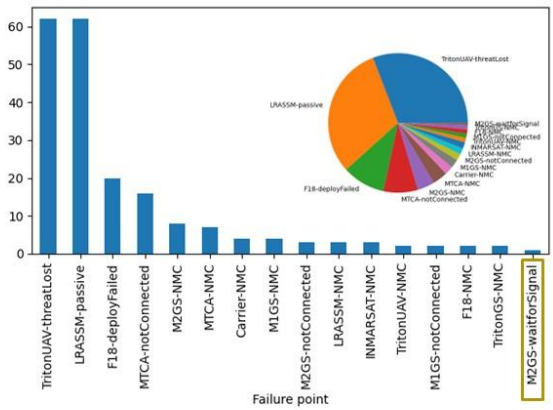
Comparing Impact of Visible Satellites

- A look into how constellation composition might impact probability of kill
- # of satellites in M1/M2 that are within visible range

2 SATs : $P(Kill) = 0.427$



5 SATs : $P(Kill) = 0.563$



44

Comparison of Demo Use Case Scenarios

- Connecting engineering decisions based on cost into consideration

Based on X_Band example mappings

	Nominal	Solar Flare (SF) (1000 SFU Event)	Mitigated w/ SF (1000 SFU Event)
Noise	Low 100 K	High 360 K	High 360 K
Power (~Cost)	Med. 10 dBW	Med. 10 dBW	High 15 dBW
Expected Latency	Low 30.12 ms	High 23,041 ms	Low 30.12 ms

System_Noise_Temp

Base Model

Lumped Model

System normally performs well

Increased noise drastically increases time to transmit

Slight increase in power/cost can mitigate noise losses

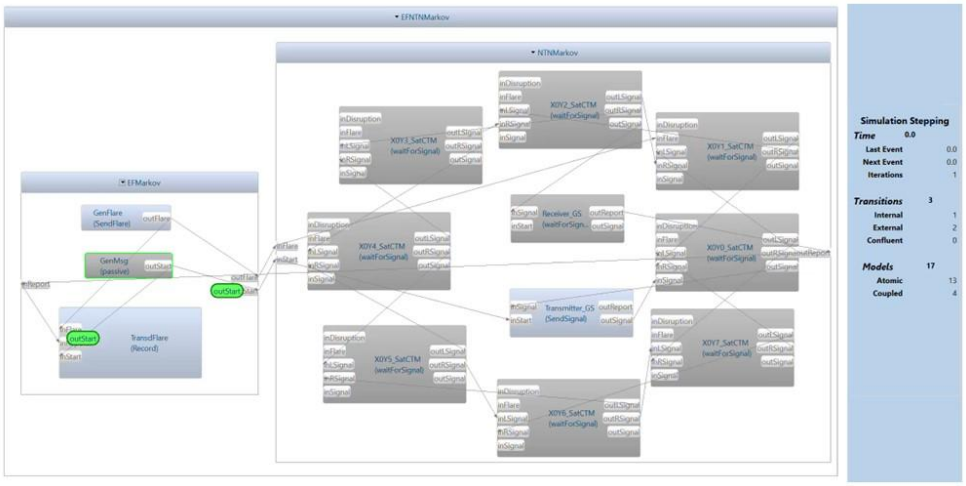
NATIONAL SECURITY INSTITUTE VIRGINIA TECH

RTSync

45

Task 4: Feasibility Demonstration (4/5)
Demo of Solar Flare Markov Simulation: Effect on End-to-end Latency

- MS4 Me SimViewer



NATIONAL SECURITY INSTITUTE VIRGINIA TECH

RTSync

46

Core Technology - What makes DEVS and MS4 Me Software different from every other modeling & simulation environments?



Math System Theory Foundation – supporting application

- Well founded computational basis for system theory-based modeling and simulation
- Mathematically-based theory helps ensure reliable system implementation and accurate simulation time management
- Hierarchical Construction supports complex system development
- Supports correct cyber-physical system modeling and simulation

In Application to Systems of Systems (SoS)

- Models, simulators and experimental frames are distinct entities
- Precise and well-defined mathematical representation
- Fosters reusable model/experiment components in repositories
- Event-based execution enables efficient computation
- DEVS software deploys on multiple single/distributed computing platforms

Discrete Event System Specification (DEVS) :
https://en.wikipedia.org/wiki/DEVS

RTSync

47

DEVS Definitions

- **DEVS Atomic Model**: This refers to the basic building block within the Discrete Event System Specification (DEVS) formalism, specified with state transition functions, an output function, a time advance function, and sets of input values, output values, and states.
- **DEVS Coupled Model**: This refers to a DEVS model that is composed of other DEVS models. It explains how to connect multiple component models to form a new model. This new model can then be used as a component in a larger coupled model, allowing for hierarchical construction.
- **DEVS Base Model**: This refers to the detailed atomic or coupled DEVS model with full structure and state transitions
- **DEVS Lumped Model**: This refers to a DEVS model in aggregated version that combines multiple components or states into fewer ones.
- **Morphism**: This refers to a formal mapping between two models that preserves their behavior. It allows you to relate:A base model (detailed, fine-grained)To a lumped model (simplified, aggregated)This mapping ensures that the lumped model behaves consistently with the base model, even though it has fewer components or states.

- **Discrete Event Simulation (DES):** Models the operation of a system as a (discrete) sequence of events in time. Each event occurs at a particular instant in time and marks a change of state in the system. [10] Between consecutive events, the model assumes no change in the system to occur.
- **Modeling and Simulation Framework (MSF):** The framework defines entities and their relationships that are central to the M&S enterprise.
- **Experimental Frame (EF):** A modeling and simulation framework used to specify the conditions under which we test and experiment our model.

NATIONAL SECURITY INSTITUTE
VIRGINIA TECH

RTSync

48